

INFORME FINAL — ERSO 2026-006

- [INFORME FINAL — ERSO 2026-006](#)
 - [Publicación del DID Document institucional](#)
 - [Cómo leer este informe](#)
 - [Contenido](#)
 - [Resumen ejecutivo](#)
- [PARTE I — MARCO CONCEPTUAL: los términos que necesitas](#)
 - [1. La historia en cinco minutos](#)
 - [2. Diccionario de términos](#)
 - [3. Las siete ideas madre](#)
 - [4. El método de trabajo: cómo se piensa un criterio](#)
- [PARTE II — QUÉ PIDE LA ERSO 006, EXPLICADO FRASE POR FRASE](#)
 - [5. La ficha del documento](#)
 - [6. Capacidades del proceso](#)
 - [7. Condiciones del proceso](#)
 - [8. Descripción del proceso](#)
 - [9. Qué debe hacer: los cinco pasos](#)
 - [10. Los cuatro criterios de aceptación](#)
- [PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO](#)
 - [11. Visión general](#)
 - [12. El entorno para esta ERSO](#)
 - [13. Mapa de requisitos a implementación](#)
- [PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA PASO Y CADA CRITERIO](#)
 - [14. Cómo funcionan los laboratorios](#)
 - [15. Preparación común: un identificador de prueba único](#)
 - [16. Laboratorio A — Paso 1: verificar las precondiciones](#)
 - [17. Laboratorio B — Paso 2: construir el documento institucional](#)
 - [18. CRITERIO 1 — El DID Document institucional se sirve por HTTPS y es resoluble](#)
 - [19. CRITERIO 2 — La escritura está autenticada y trazada, limitada al namespace de la entidad](#)
 - [20. CRITERIO 3 — El documento no expone material de claves privadas ni datos civiles](#)
 - [21. CRITERIO 4 — La evidencia de publicación \(versión, hash y URL pública\) queda registrada](#)
 - [22. Pruebas automáticas y recorrido de extremo a extremo](#)
- [PARTE V — PREGUNTAS Y RESPUESTAS](#)
 - [23. Preguntas sobre los conceptos](#)
 - [24. Preguntas por laboratorio y por criterio](#)
- [PARTE VI — CÓMO RESPONDER A CADA CRITERIO](#)
 - [25. Redacción modelo de las cuatro respuestas](#)
 - [26. Lista de comprobación para quien acepta](#)
- [PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN](#)
 - [27. Lo que este informe NO demuestra, y lo que se encontró](#)
 - [28. Si algo no sale como en el informe](#)
 - [29. Operación diaria](#)
- [ANEXOS](#)
 - [Anexo A — Los scripts](#)
 - [Anexo B — Autocomprobación \(con respuestas\)](#)
 - [Anexo C — Ejercicios del método \(con respuesta modelo\)](#)

INFORME FINAL — ERSO 2026-006

Publicación del DID Document institucional

Manual de comprensión, construcción y comprobación

Proyecto	Identidad Digital Soberana SSI
ERSo	2026-006 — Publicación del DID Document institucional
Desarrollador asignado (según la ERSo)	Luis González
Plantilla de pruebas y pruebas funcionales (según la ERSo)	Geovani Rincón
Fechas de la ERSo	Elaboración: viernes 18-sep-2026 · Desarrollo: miércoles 30-sep-2026, 7:30 a. m. · Prueba: viernes 2-oct-2026, 7:30 a. m.
Dominio del laboratorio	<code>civica-desarrollo.avance.org.co</code>
Entidad de ejemplo	<code>avance-issuer</code> (Avance, emisor institucional) · espacio <code>entidades/avance</code>
Ubicación del proyecto	<code>/home/geovani/Descargas/generic/bitacora/new_proyect</code>
Salidas de terminal de este informe	Obtenidas ejecutando los comandos reales sobre el proyecto, el 1 de octubre de 2026
Complementos	<code>scripts/lab-006-completo.sh</code> (todos los laboratorios en un script) · <code>informes/ERSo-2026-006.md</code> (versión corta) · <code>informes/INFORME-FINAL-ERSo-2026-004.md</code> y <code>...-005.md</code> (el registro y la publicación sobre los que se apoya esta ERSo)

Cómo leer este informe

Este documento está escrito para que **cualquier persona** pueda entender qué se pidió, qué se construyó y cómo se comprobó, **sin necesidad de ser experta**. Cada término técnico se explica la primera vez que aparece; la Parte I es un diccionario de esos términos.

Símbolo	Significa
	Idea clave: lo más importante de la sección, en una frase
	Qué vas a ver: lo que debes esperar <i>antes</i> de ejecutar un comando
	Qué significa: cómo interpretar lo que el sistema respondió
	Cuidado: una trampa frecuente o un límite que conviene conocer
	Predice antes de ejecutar: una pregunta para que pienses antes de ver el resultado

Si eres...	Lee esto
Quien decide o aprueba	El Resumen ejecutivo y la Parte VII
Quien necesita entender el problema	Parte I y Parte II
Quien debe reproducir y comprobar	Partes III y IV
Quien prepara la prueba funcional del 2 de octubre	Partes IV, VI y la lista de comprobación (§26)
Quien prepara una exposición	Partes V y VI

Cómo están presentados los comandos. Los bloques `console` son una **sesión de terminal real**: las líneas con `$` son lo que se escribe; las demás, lo que el sistema respondió. Los valores que cambian en cada ejecución (horas, identificadores, claves, huellas) serán distintos al repetir; lo que debe coincidir es el **patrón** explicado debajo de cada salida.

Esta ERSO se apoya en las dos anteriores. La 004 construyó el registro (VDR) y la 005 publicó documentos de *laboratorio*. Esta ERSO hace lo mismo pero con un sujeto distinto: **la institución**. Todo lo necesario para seguirla sola se explica aquí.

Contenido

Parte	Secciones	De qué trata
Resumen ejecutivo	—	El problema, la solución y el resultado en una página
I · Marco conceptual	1 a 4	Los términos, en lenguaje sencillo, y el método de trabajo
II · Qué pide la ERSO	5 a 10	Capacidades, condiciones, descripción, pasos y criterios, frase por frase
III · El proyecto	11 a 13	Qué se construyó para esta ERSO, cómo está montado en Docker y cómo funciona por dentro
IV · Laboratorios	14 a 22	La prueba material de cada paso y criterio, con cada comando y su respuesta
V · Preguntas y respuestas	23 y 24	Todas las preguntas de comprensión con su respuesta
VI · Cómo responder	25 y 26	Redacción modelo de cada criterio y lista de comprobación
VII · Límites y operación	27 a 29	Lo que NO se demuestra, qué hacer si algo falla y operación diaria
Anexos	A a D	Scripts, autocomprobación, ejercicios y referencias

Resumen ejecutivo

El problema. Para que alguien confíe en una credencial, necesita saber **quién la emitió** y poder comprobar su firma. Eso exige que la **institución emisora** (Avance) tenga una identidad pública verificable: un documento que diga «esta es mi clave pública, y esta es mi dirección de servicio». Pero un documento así es un blanco: si cualquiera pudiera escribir o modificar el de la institución, podría **suplantarla**.

La solución. La institución publica su propio *DID Document* en un espacio de nombres exclusivo (entidades/...). **Escribir** exige tres cosas a la vez: un certificado de cliente (mTLS), unas credenciales de servicio que producen un token con permiso sobre ese espacio, y una prueba de posesión de la clave del propio DID. **Leer**, en cambio, es público y va por **otra puerta** que no tiene ninguna forma de escribir. Cada publicación deja evidencia: versión, hash y URL pública.

El resultado.

#	Criterio	Resultado	Dónde se ve
1	El documento institucional se sirve por HTTPS y es resoluble	✓	\$18
2	La escritura está autenticada y trazada, limitada al espacio de la entidad	✓	\$19
3	No expone claves privadas ni datos civiles	✓	\$20
4	La evidencia de publicación (versión, hash y URL) queda registrada	✓	\$21

Cifras: 6 pruebas automáticas propias de esta ERSO (todas pasan); el recorrido de extremo a extremo completo del proyecto (121 comprobaciones) también está en verde.

Lo que debes saber de antemano: (1) para este informe fue necesario **reservar un espacio comodín** entidades/* a Avance, para poder repetir los laboratorios sin ensuciar el DID real de la institución (§12.5); (2) los intentos con un certificado de una CA desconocida los frena nginx, **antes** de la aplicación, así que no dejan huella en la auditoría de la aplicación (§19.4).

PARTE I — MARCO CONCEPTUAL: los términos que necesitas

1. La historia en cinco minutos

Imagina un **notario** (la institución emisora) que firma diplomas. Quien reciba un diploma querrá saber: «¿este sello es realmente del notario?». Necesita una **hoja pública** del notario con su sello oficial (su **clave pública**) y la dirección de su oficina.

- Esa hoja es el **DID Document institucional**.
- Para que sea confiable, **solo el notario** puede modificarla (**escritura autenticada**), pero **cualquiera** puede leerla (**lectura pública**).
- Para escribir, el notario entra por una **puerta de servicio** donde le piden tres cosas: su carné con foto (**certificado de cliente**), su usuario y contraseña (**credenciales de servicio**) y que demuestre que tiene el sello (**prueba de posesión**).
- La hoja está en una **repisa con su nombre** (el **espacio de nombres** entidades/avance); ninguna otra entidad puede escribir en esa repisa.
- Cada vez que se publica, un **libro de registro** apunta quién, cuándo, qué versión y qué huella tenía (**evidencia**).

💡 Esta ERSO trata de que la **identidad de la institución** sea pública, comprobable y **solo modificable por ella**.

2. Diccionario de términos

2.1 Los de siempre (resumen)

Estos términos ya se explicaron en los informes 004 y 005; aquí van en una línea para que este documento se baste solo.

Término	En palabras sencillas
DID	Un identificador con la forma <code>did:web:dominio:ruta</code> . No es un número de cédula; es una <i>dirección</i> que se puede calcular.
<code>did:web</code>	Método de DID donde el identificador se convierte en una URL web: <code>did:web:ejemplo.org:entidades:avance</code> → <code>https://ejemplo.org/entidades/avance/did.json</code> .
DID Document	El archivo JSON público de un DID: dice quién es el DID, qué claves públicas tiene y para qué sirven.
Clave pública / privada	Par matemático: la privada firma y nunca se comparte ; la pública verifica y puede ser pública .
P-256	El tipo de curva elíptica de las claves usadas aquí.
Multikey	Formato estándar para escribir una clave pública como texto (<code>zDn...</code>).

Término	En palabras sencillas
Hash (sha256:...)	Huella digital de un contenido. Cambia por completo si se altera un solo carácter.
JSON canónico	El JSON escrito siempre igual (claves ordenadas, sin espacios) para que el hash sea estable.
VDR	<i>Registro verificable de datos</i> : el servicio que guarda y entrega los DID Document. Se construyó en la ERSo 004.
Versión	Cada cambio de un documento crea una versión nueva; las anteriores no se borran.
Auditoría	Libro de registro de lo ocurrido, que no se puede editar ni borrar.

2.2 Los canales: dos puertas distintas

Término	En palabras sencillas
Canal	Un camino por el que entra el tráfico: una dirección + un puerto + unas reglas.
Canal público de lectura	La puerta ancha: cualquiera puede <i>leer</i> documentos. Es el puerto HTTPS normal (443 dentro; 8443 en este equipo). Solo admite GET (y HEAD) sobre rutas <code>.../did.json</code> .
Canal de escritura	La puerta de servicio: solo entra quien tenga certificado de cliente. Dentro es el puerto 8443; en este equipo se alcanza por el 9443 .
Separación de canales	Que las dos puertas sean <i>distintas</i> y no se puedan confundir: ni se escribe por la pública ni se lee por la de escritura.
nginx	El «portero» (proxy inverso) que recibe todo el tráfico, aplica las reglas de cada puerta y reenvía a la aplicación.
Proxy inverso	Un servidor que se pone delante de otro y recibe las peticiones en su nombre.
Certificado de servidor	El que prueba al cliente que <i>está hablando con el servidor correcto</i> (es lo que hace «HTTPS»).
Certificado de cliente	El que prueba al servidor <i>quién es el cliente</i> . Como un carné con foto.
mTLS (TLS mutuo)	Conexión segura donde los dos lados muestran certificado. El servidor verifica que el del cliente lo firmó la autoridad (CA) de confianza.
CA (autoridad certificadora)	Quien firma certificados. Aquí es propia del laboratorio (<code>ca.crt</code>).
CN (nombre común)	El nombre que va escrito en el certificado. Aquí coincide con el identificador de la entidad: <code>avance-issuer</code> .
Cabeceras X-SSL-Client-*	Mensajes que nginx añade al reenviar la petición para decir a la aplicación «el certificado del cliente era válido y de tal CN». nginx las sobrescribe siempre , para que nadie las falsifique.

2.3 Quién puede escribir

Término	En palabras sencillas
Cuenta de entidad	El registro de una organización autorizada a usar el VDR (<code>avance-issuer</code> , <code>lab-operator</code> ...). Puede estar habilitada o no.

Término	En palabras sencillas
OAuth2 client credentials	Forma estándar de que un <i>servicio</i> (no una persona) se identifique: envía <code>client_id</code> y <code>client_secret</code> y recibe un token.
Token de acceso (JWT)	Un pase firmado que dura poco (10 minutos). Dice quién es la entidad y qué puede hacer.
Scope (permiso)	Lo que el token permite. Aquí: <code>did:write:<espacio></code> , por ejemplo <code>did:write:entidades/*</code> .
Espacio de nombres (namespace)	La «repisa» reservada: una ruta (<code>entidades/avance</code>) que pertenece a una sola entidad. Nadie más puede escribir ahí.
Comodín de un nivel (entidades/*)	Reserva <i>todas</i> las rutas de un nivel bajo <code>entidades/</code> : <code>entidades/x</code> , <code>entidades/y</code> ... pero no <code>entidades/x/y</code> .
Ligadura certificado↔token	Regla: el CN del certificado debe ser la misma entidad que la del token. Un token robado no sirve con otro certificado.
Desafío (challenge)	Un número aleatorio, de un solo uso y con vencimiento, que el servidor entrega para que el cliente demuestre que <i>posee la clave privada</i> .
Prueba de posesión	Firma (JWS) del cliente sobre el desafío y el hash del documento, hecha con la clave del propio DID.
Precondición	Algo que debe ser cierto <i>antes</i> de empezar: cuenta habilitada, espacio propio, DID válido. Si falla, ni siquiera se entrega un desafío .
Control de concurrencia (If-Match)	Para modificar, hay que decir qué versión se cree tener (<code>0</code> = no existe). Evita pisar cambios ajenos.

2.4 Evidencia y trazabilidad

Término	En palabras sencillas
Operación	Cada escritura queda como una <i>operación</i> con identificador, propósito, estado, versión, hash y URL.
Confirmación	Tras escribir, el registro vuelve a leer la URL pública y compara hash. Si coincide: <code>CONFIRMED</code> ; si no se pudo comprobar: <code>PENDING</code> .
Evidencia de publicación	El conjunto <i>versión + hash + URL pública</i> de una operación. Es lo que el criterio 4 pide conservar.
Trazabilidad	Poder reconstruir quién hizo qué, cuándo y desde dónde .
Entidad institucional	Emisor (quien firma credenciales) o ancla de confianza (quien respalda a otros). No es una persona.

3. Las siete ideas madre

1. **Una institución también es una identidad:** necesita un DID Document propio.
2. **Leer es público; escribir es privilegiado:** y son dos puertas distintas.
3. **Escribir exige tres pruebas:** certificado (quién llega), token (qué puede hacer) y firma del propio DID (que posee la clave).
4. **Cada entidad tiene su repisa:** no se puede escribir en la de otro.
5. **Se valida antes de actuar:** una petición ajena ni siquiera recibe desafío.
6. **El documento solo lleva lo público:** nada de claves privadas ni datos civiles.

7. **Todo deja rastro:** operación + auditoría, que no se editan.

4. El método de trabajo: cómo se piensa un criterio

1. **Descomponer** el criterio en el verbo y la evidencia que pide.
 2. **Hacerlo comprobable:** ¿qué comando o consulta lo demuestra?
 3. **Buscar el caso negativo:** un criterio de seguridad se prueba también con lo que **debe fallar**.
 4. **Predecir** el resultado antes de ejecutar.
 5. **Ejecutar y observar** el resultado real.
 6. **Correlacionar** con la regla que lo causa.
 7. **Redactar** la respuesta con la evidencia.
-

PARTE II — QUÉ PIDE LA ERSo 006, EXPLICADO FRASE POR FRASE

5. La ficha del documento

Campo del PDF	Valor
Proyecto	Identidad Digital Soberana SSI
Título	Publicación del DID Document institucional
Desarrollador	Luis González
Fecha de la ERSo	Viernes, 18 de septiembre de 2026
Fecha de desarrollo	Miércoles, 30 de septiembre de 2026, 7:30 a. m.
Fecha de prueba	Viernes, 2 de octubre de 2026, 7:30 a. m.
Responsables	Análisis y diseño, asignar desarrollador, aceptar archivos y archivo: Rocío Villamizar · Implementar: Luis González · Plantilla de pruebas y pruebas funcionales: Geovani Rincón

⚠ Las firmas y fechas de la tabla de actividades **corresponden a las personas responsables**; este informe aporta las **evidencias técnicas**, no las firmas.

Capacidades → Condiciones → Descripción → Qué debe hacer (5 pasos) → Criterios (4) → Actividades

6. Capacidades del proceso

6.1 La frase principal, desarmada

"Se requiere habilitar la **publicación autenticada** del DID Document correspondiente a la **entidad institucional** (emisor o ancla de confianza), con **namespace propio**, y dejar **evidencia verificable** de la publicación."

#	Fragmento	En lenguaje sencillo	Cómo se materializó
1	" publicación autenticada "	No cualquiera publica: hay que probar quién se es	mTLS + token + prueba de posesión
2	" entidad institucional (emisor o ancla de confianza) "	El sujeto es una organización, no un ciudadano	Cuenta avance-issuer

#	Fragmento	En lenguaje sencillo	Cómo se materializó
3	"namespace propio"	Una ruta reservada solo para esa entidad	entidades/avance (y comodín entidades/*)
4	"evidencia verificable"	Que se pueda demostrar después qué se publicó	Operación + hash + auditoría

6.2 Las cinco capacidades

#	Capacidad (texto del PDF)	Qué se busca	Cómo se resolvió	Paso
1	Construir el DID Document institucional con claves públicas en Multikey	Un JSON con solo material público , más (opcional) el servicio de emisión	DidDocumentBuilder.build (-- service-url)	2
2	Publicar en la ruta web calculada y servirlo por HTTPS	Que quede en la dirección que el DID dicta	Registro + nginx	3
3	Ejecutar la publicación mediante canal de escritura autenticado entre servicios	Escritura solo de servicio a servicio, con credenciales fuertes	mTLS + OAuth2 + prueba de posesión	3
4	Confirmar leyendo la URL pública y comparando contenido y hash	No creerse a uno mismo: leer lo publicado como un tercero	Confirmación automática CONFIRMED / PENDING	4
5	Conservar la evidencia (versión, hash y URL pública)	Un registro que no se pueda alterar	Tabla operations + audit_log	5

7. Condiciones del proceso

Una **condición** es una restricción que el diseño debe respetar siempre. Para cada una: **qué problema evita**.

Condición 1 — "La publicación la ejecuta la entidad institucional con escritura autenticada."

- **Qué significa.** Quien publica es la propia entidad, y se identifica de forma fuerte.
- **Qué problema evita.** Que un tercero (o un administrador descuidado) publique en nombre de la institución.
- **Cómo se cumple.** Tres capas: certificado cliente, token con permiso y firma con la clave del DID (§19).

Condición 2 — "El namespace institucional es propio y no colisiona con el del titular."

- **Qué significa.** La repisa entidades/... es de la institución; la del ciudadano (titulares/...) es otra, y no se mezclan.
- **Qué problema evita.** Que una entidad escriba en el espacio de un titular, o al revés; y la suplantación entre entidades.
- **Cómo se cumple.** Cada espacio tiene un único dueño en la tabla namespaces ; si no eres el dueño, no hay desafío (NAMESPACE_NOT_OWNED).

Condición 3 — "La ruta `did:web` institucional se encuentra reservada en el dominio contratado."

- **Qué significa.** Antes de publicar, la ruta ya está apartada para esa entidad, dentro del dominio propio.
- **Qué problema evita.** Que dos entidades se pisen la misma dirección, o que se publique en un dominio ajeno.
- **Cómo se cumple.** Reserva al arrancar (`VDR_CLIENTS`) y comprobación del dominio (`DID_DOMAIN_MISMATCH`).

Condición 4 — "La lectura pública se realiza por un canal separado del canal de escritura."

- **Qué significa.** Dos puertas. La de lectura no tiene forma de escribir.
- **Qué problema evita.** Que un fallo o ataque en la lectura (la más expuesta) permita modificar documentos.
- **Cómo se cumple.** Dos puertos en nginx y **dos servidores** distintos dentro de la aplicación (8080 público, 8081 administración).

Condición 5 — "El documento no incluye claves privadas, credenciales personales ni datos civiles."

- **Qué significa.** Solo material público.
- **Qué problema evita.** Filtrar secretos o datos personales en un documento que todo el mundo puede leer.
- **Cómo se cumple.** Validador con **lista blanca** (§20).

8. Descripción del proceso

"Aquí el sujeto que publica no es una persona, sino la entidad institucional..."

Es la diferencia con la ERSO 005: allí se publicaba un DID de laboratorio; aquí lo hace **la institución real** en su propio espacio.

"...que expone su propio DID Document con un espacio de nombres exclusivo."

«Exclusivo»: nadie más escribe ahí.

"El propósito es que esa entidad tenga identidad verificable publicada, con escritura autenticada y lectura pública separadas, y con evidencia de cada versión publicada."

Cuatro propósitos: identidad **verificable**, escritura **autenticada**, lectura **separada**, evidencia **por versión**.

"Literatura y temas a consultar"

Lectura	Qué te aporta
W3C DID ^s v1.1	Qué lleva un DID Document
Método did:web	La regla DID → URL
Multikey / multibase	Cómo se escribe la clave pública
mTLS y OAuth2 client credentials	El canal de escritura
API de documentos gestionados con control de versión	If-Match , versiones, operaciones

Enlaces en `docs/VERSIONES-NORMATIVAS.md`.

9. Qué debe hacer: los cinco pasos

1. Verificar las precondiciones: cuenta de entidad habilitada con namespace propio y ruta did:web reservada.
2. Construir el DID Document institucional con las claves públicas de la entidad en Multikey.
3. Publicar el documento por el canal de escritura autenticado (mTLS y token de servicio con permiso sobre el namespace).
4. Confirmar la publicación leyendo el documento desde la URL calculada y comparando contenido y hash.
5. Registrar la evidencia de publicación (versión, hash y URL pública).

Paso	Lo que pide	Criterio que lo comprueba	Laboratorio
1	Precondiciones	Criterio 2 (límite al espacio de la entidad)	§16
2	Construir el documento	Criterio 3 (solo material público)	§17
3	Publicar por canal autenticado	Criterios 1 y 2	§18 y §19
4	Confirmar leyendo la URL	Criterio 1	§18
5	Registrar la evidencia	Criterio 4	§21

💡 Los criterios no están en el mismo orden que los pasos: el criterio 2 (autenticación) se prueba en el paso 3 y en el paso 1, porque **las precondiciones son parte de la seguridad**.

10. Los cuatro criterios de aceptación

Criterio 1. "El DID Document institucional se sirve por HTTPS y es resoluble; evidencia: lectura desde la URL pública." **Criterio 2.** "La escritura está autenticada y trazada, limitada al namespace de la entidad; evidencia: registro de la operación." **Criterio 3.** "El documento no expone material de claves privadas ni datos civiles; evidencia: revisión del contenido." **Criterio 4.** "La evidencia de publicación (versión, hash y URL pública) queda registrada; evidencia: registro de publicación."

Criterio	Verbo y evidencia	Lo que debo poder mostrar	El caso negativo	Mecanismo
1	<i>Se sirve y es resoluble</i> · lectura pública	GET HTTPS devuelve el documento, y un cliente DID lo resuelve (devuelve documento + metadatos)	Otra ruta/verbo no lo sirven	nginx + ruta pública + <code>DidWebResolver</code>
2	<i>Autenticada, trazada, limitada</i> · registro de la operación	Escritura exitosa con actor identificado en auditoría	Contraseña mala, sin certificado, certificado ajeno, espacio ajeno	mTLS + OAuth2 + ligadura + namespaces
3	<i>No expone</i> · revisión del contenido	Solo las propiedades autorizadas, sin secretos	Publicar <code>privateKeyMultibase</code> , <code>credentialSubject</code> , controlador ajeno	Lista blanca
4	<i>Queda registrada</i> · registro de publicación	Operación con versión, hash, URL, <code>confirmedAt</code>	El hash registrado ≠ el del documento público (no debe ocurrir)	<code>operations</code> + <code>audit_log</code>

⚠ **Dos términos del criterio 1 que conviene distinguir:** «se sirve» (el servidor lo entrega) y «es resoluble» (un *cliente DID* —aquí el resolutor de la ERSO 007— puede convertir el DID en documento y validarlo). Se prueban por separado en §18.

PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO

11. Visión general

11.1 Qué se construyó (y qué no)

La ERSO 006 **no añade un servicio nuevo**. Reutiliza el VDR de la ERSO 004 y el flujo de publicación de la 005, y les añade lo que hace falta para que quien publique sea **una institución**:

Pieza	Nueva en esta ERSO	Qué aporta
Cuenta <code>avance-issuer</code> + sus espacios	Sí (configuración)	La identidad de servicio de la institución
Certificado <code>avance-issuer.p12</code>	Sí (generado por <code>gen-dev-certs.sh</code>)	El «carné» de la institución para el canal de escritura
Servicio <code>issuer</code> en el documento	Opción <code>--service-url</code> de <code>build-doc</code>	Publicar dónde se emiten credenciales (OID4VCI)
Ligadura certificado↔token	Reutilizada de la 004	Un token robado no sirve con otro certificado
<code>GET /admin/v1/operations/{id}</code>	Reutilizada	La evidencia del criterio 4
Pruebas <code>Erso006InstitutionalTest</code>	Sí	6 pruebas automáticas
Espacio comodín <code>entidades/*</code>	Sí (configuración, para los laboratorios)	Repetir ejercicios sin tocar el DID real (§12.5)

11.2 Las piezas en Docker

El proyecto corre en **Docker Compose**. Cada pieza es un contenedor:

Contenedor	Para qué sirve en esta ERSO
<code>nginx</code>	El portero: dos puertas (lectura y escritura) y la comprobación del certificado cliente
<code>vdr</code>	La aplicación del registro: dos servidores internos (público 8080, administración 8081)
<code>postgres</code>	La base de datos: cuentas, espacios, documentos, versiones, operaciones, auditoría
<code>wallet</code> , <code>credential</code>	Otros servicios del proyecto; no intervienen aquí, pero deben seguir funcionando (camino base)
<code>tools</code> (perfil <code>tools</code>)	Una caja de herramientas (<code>did-tools</code>) que se ejecuta <i>bajo demanda</i> para firmar y escribir como lo haría un servicio cliente

Esto significa: **nunca se abre la base de datos ni la aplicación al exterior**; solo `nginx` publica puertos.

```
$ docker compose ps --format 'table {{.Service}}\t{{.Ports}}'
SERVICE      PORTS
credencial
nginx          80/tcp, 0.0.0.0:8444->8444/tcp, [::]:8444->8444/tcp, 0.0.0.0:8443->443/tcp, [::]:8443->443/tcp, 0.0.0.0:9443->8443/tcp, [::]:9443->8443/tcp
postgres      5432/tcp
vdr
wallet
```

✓ **Cómo leerlo:** `nginx` publica tres puertos al equipo: **8443→443** (lectura pública), **9443→8443** (escritura con certificado cliente) y **8444** (cartera/credenciales). Los demás servicios no publican nada.

11.3 El recorrido de una publicación institucional

```
Servicio de Avance (did-tools write)
| 1. TLS mutuo: muestra su certificado avance-issuer
▼
nginx :9443 (escritura) — verifica el certificado con la CA — añade X-SSL-Client-*
| 2. POST /oauth/token (client_id + secreto) → JWT con scope did:write:entidades/...
| 3. POST /challenges ({precondiciones OK?}) → desafío de un solo uso
| 4. PUT /documents/{did} + prueba firmada con la clave del DID
▼
vdr :8081 — valida (lista blanca) — guarda versión + operación + auditoría
| 5. lee su propia URL pública y compara hash → CONFIRMED
▼
cualquiera → nginx :8443 (lectura) → vdr :8080 → did.json
```

12. El entorno para esta ERSO

12.1 Entrar a la carpeta y preparar el terminal

Todos los laboratorios se ejecutan desde `deploy/`, con las variables del archivo `.env` cargadas. Se definen atajos para no repetir comandos largos:

Atajo	Qué hace
D	El dominio (<code>civica-desarrollo.avance.org.co</code>)
PUB	Opciones de <code>curl</code> para la puerta pública (puerto 8443), confiando en nuestra CA
ADM	Opciones de <code>curl</code> para la puerta de escritura (puerto 9443)
PSQL	Entra a la base de datos para consultar
TOOLS	Ejecuta <code>did-tools</code> en su contenedor
ESCRIBIR	<code>TOOLS write</code> con la identidad (usuario, secreto y certificado) que se le indique
ADMINISTRAR	<code>TOOLS admin</code> con la identidad que se le indique

⚠ **ESCRIBIR** y **ADMINISTRAR** reciben **tres identidades distintas**: el `client_id`, el secreto y el certificado. Poder combinarlas libremente es lo que permite, en el criterio 2, probar «usuario de uno con carné de otro».

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyector/deploy
$ set -a; . ./env; set +a; export LOCAL_UID=$(id -u) LOCAL_GID=$(id -g)
$ D=$VDR_DOMAIN
$ PUB="--cacert certs/ca.crt --resolve $D:8443:127.0.0.1"
$ ADM="--cacert certs/ca.crt --resolve $D:9443:127.0.0.1"
$ W=../evidencias/work
$ PSQL="docker compose exec -T postgres psql -U $POSTGRES_USER -d $POSTGRES_DB"
$ TOOLS() { docker compose --progress quiet --profile tools run --rm -T tools "$@"; }
$ ESCRIBIR() { local cid=$1 sec=$2 p12=$3 did=$4; shift 4; TOOLS write --admin-url "https://$D:8443" --client-id "$cid" --secret "$sec" --p12 "/certs/$p12.p12" --p12-pass "$P12_PASS" --ca /certs/ca.crt --did "$did" "$@"; }
$ ADMINISTRAR() { local cid=$1 sec=$2 p12=$3; shift 3; TOOLS admin --admin-url "https://$D:8443" --client-id "$cid" --secret "$sec" --p12 "/certs/$p12.p12" --p12-pass "$P12_PASS" --ca /certs/ca.crt "$@"; }
```

12.2 Comprobar que el registro está vivo

```
$ docker compose ps --format 'table {{.Service}}\t{{.Status}}'
SERVICE      STATUS
credential    Up 5 minutes
nginx          Up 5 minutes
postgres       Up 5 minutes (healthy)
vdr            Up 5 minutes
wallet         Up 5 minutes
$ curl -s $PUB https://$D:8443/health
{"status":"UP","vdr":"enabled"}
```

✓ Los cinco servicios están **Up** y la base **healthy**; `{"status":"UP","vdr":"enabled"}` confirma que la extensión está encendida (si estuviera apagada, no se podría hacer nada de esta ERSO).

12.3 Las dos puertas de nginx

La puerta pública (lectura):

```
$ sed -n '/listen 443/,/location \/{ return 404; }/p' nginx/nginx.conf
listen 443 ssl;
server_name civica-desarrollo.avance.org.co localhost;
ssl_certificate /etc/nginx/certs/server.crt;
ssl_certificate_key /etc/nginx/certs/server.key;

add_header X-Content-Type-Options nosniff always;

location = /health { proxy_pass http://vdr:8080; }
location = /base/ping { proxy_pass http://vdr:8080; }

location ~ /did\.json$ {
    limit_except GET HEAD { deny all; }
    proxy_pass http://vdr:8080;
}

location / { return 404; }
```

- `listen 443 ssl` — HTTPS con el certificado del servidor, **sin pedir** certificado al cliente.
- `location ~ /did\.json$` — solo se atienden rutas que terminan en `did.json`.
- `limit_except GET HEAD { deny all; }` — cualquier verbo distinto de `GET/HEAD` se rechaza.
- `location / { return 404; }` — todo lo demás no existe.

La puerta de escritura:


```
$ sed -n '/listen 8443/,/location \/{ return 404; }/p' nginx/nginx.conf
listen 8443 ssl;
server_name civica-desarrollo.avance.org.co localhost;
ssl_certificate /etc/nginx/certs/server.crt;
ssl_certificate_key /etc/nginx/certs/server.key;

ssl_client_certificate /etc/nginx/certs/ca.crt;
ssl_verify_client on;

# Portales de administración (Issuer/Verifier Admin Portal): solo por este canal, además
exigen su token.
location ~ ^/admin/(offers|verifier/requests|execution-log) {
    set $credential_upstream http://credential:8100;
    proxy_pass $credential_upstream;
}

location /admin/v1/ {
    # Se SOBREScriben siempre: un cliente no puede falsificar su identidad con encabezados
    propios.
    proxy_set_header X-SSL-Client-Verify $ssl_client_verify;
    proxy_set_header X-SSL-Client-S-DN $ssl_client_s_dn;
    proxy_pass http://vdr:8081;
}

location / { return 404; }
```

- `ssl_verify_client on` — **exige** certificado de cliente firmado por nuestra CA.
- `proxy_set_header X-SSL-Client-Verify ...` y `X-SSL-Client-S-DN` — nginx **siempre** escribe estas cabeceras con lo que él comprobó; un cliente no puede enviarlas por su cuenta y salirse con la suya.
- `location /admin/v1/ { proxy_pass http://vdr:8081; }` — solo se reenvían las rutas administrativas.

💡 Observa que la puerta de escritura **no tiene** ninguna ruta `did.json`: por eso, leer por ahí da 404 (\$19.6).

12.4 Las cuentas y espacios configurados

Las cuentas y sus espacios se declaran en la variable `VDR_CLIENTS` (sin los secretos, que se omiten aquí):

```
$ echo "$VDR_CLIENTS" | jq 'map(del(.secret, .clientSecret) |
with_entries(select(.key|test("secret";"i")|not)))'
[
  {
    "clientId": "avance-issuer",
    "displayName": "Avance (emisor institucional)",
    "namespaces": [
      "entidades/avance",
      "entidades/avance-ciclo",
      "entidades/*"
    ]
  },
  {
    "clientId": "lab-operator",
    "displayName": "Laboratorio de pruebas de identidad",
    "namespaces": [
      "lab/laboratorio",
      "lab/*"
    ]
  }
],
```

```
{
  "clientId": "vdr-admin",
  "displayName": "Administración VDR",
  "admin": true
},
{
  "clientId": "wallet-backend",
  "displayName": "Wallet Backend",
  "namespaces": [
    "titulares/*"
  ]
}
]
```

Al arrancar, el registro **vuelca** esa configuración en las tablas `entity_accounts` y `namespaces`.

12.5 El espacio comodín `entidades/*`, y por qué se añadió

⚠ **Esto es un cambio de configuración que conviene entender.** Originalmente, Avance solo tenía `entidades/avance` y `entidades/avance-ciclo`. Para el criterio 1 basta con el DID real de Avance. Pero los laboratorios de este informe **publican documentos nuevos cada vez** (para poder repetir el ejercicio), y no es buena práctica ensuciar ni modificar el DID real de la institución para hacerlo.

Solución: reservar a Avance un espacio **comodín de un nivel** `entidades/*` (de la misma forma que el laboratorio ya tenía `lab/*`). Con él, los ejercicios publican en `entidades/inst-006-<hora>`; el DID real `entidades/avance` no se toca. Se aplicó en `deploy/.env` y en la plantilla `scripts/gen-env.sh`.

```
$ echo "select client_id, display_name, enabled from entity_accounts order by 1" | $PSQL
 client_id | display_name | enabled
-----+-----+-----
 avance-issuer | Avance (emisor institucional) | t
 lab-operator | Laboratorio de pruebas de identidad | t
 vdr-admin | Administración VDR | t
 wallet-backend | Wallet Backend | t
(4 rows)
$ echo "select path, owner_client_id from namespaces where path not like '%prueba-005%' order by path" | $PSQL
 path | owner_client_id
-----+-----
 entidades/* | avance-issuer
 entidades/avance | avance-issuer
 entidades/avance-ciclo | avance-issuer
 entidades/inst-006-1790873626 | avance-issuer
 lab/* | lab-operator
 lab/laboratorio | lab-operator
 titulares/* | wallet-backend
 titulares/WI-RWVC8mA75544XCtyKnInxiAFrvKaR-r6WjmDHw6w | wallet-backend
(8 rows)
```

✓ Hay cuatro cuentas habilitadas y siete espacios. Cada espacio tiene **un solo dueño**. `entidades/*` es de `avance-issuer`; `lab/*`, de `lab-operator`; `titulares/*`, de `wallet-backend`.

12.6 Las reglas de precondition

Antes de entregar un desafío, el registro evalúa una lista de condiciones; si alguna falla, **no hay desafío**. Estas son las reglas (y su ubicación en el código):

```
$ grep -n
'NAMESPACE_NOT_OWNED\|ACCOUNT_MISSING_OR_DISABLED\|WRITE_CHANNEL_NOT_RESTRICTED\|DID_DOMAIN_MISMATCH\
|NAMESPACE_NOT_RESERVED\|DID_INVALID' ../vdr-
service/src/main/kotlin/co/org/avance/ssi/vdr/RegistryService.kt | cut -c1-170
128:         if (store.accountEnabled(c, clientId) != true) f += "ACCOUNT_MISSING_OR_DISABLED"
129:         if (!channelRestricted) f += "WRITE_CHANNEL_NOT_RESTRICTED"
130:         val id = try { adapter.parse(did) } catch (e: InvalidDidException) { f += "DID_INVALID";
return f }
131:         if (id.host != cfg.domain.lowercase()) f += "DID_DOMAIN_MISMATCH"
134:         if (owner == null) f += "NAMESPACE_NOT_RESERVED"
136:         if (owner != clientId) f += "NAMESPACE_NOT_OWNED"
174:         val id = try { adapter.parse(didStr) } catch (e: InvalidDidException) { throw
ApiException(HttpStatusCode.BadRequest, "DID_INVALID", e.message ?: "DID inváli
175:         if (id.host != cfg.domain.lowercase()) throw
ApiException(HttpStatusCode.UnprocessableEntity, "DID_DOMAIN_MISMATCH", "El DID no pertenece al
dominio ${cfg.dom
177:         db.tx { c -> if (store.namespaceOwner(c, ns) != clientId) throw denied(c, clientId,
didStr, "NAMESPACE_NOT_OWNED") }
324:         val id = try { adapter.parse(did) } catch (e: InvalidDidException) { throw
ApiException(HttpStatusCode.BadRequest, "DID_INVALID", e.message ?: "DID inválido"
326:         if (owner != clientId) throw ApiException(HttpStatusCode.Forbidden,
"NAMESPACE_NOT_OWNED", "La entidad no tiene permiso sobre este namespace")
```

Código	Significa
ACCOUNT_MISSING_OR_DISABLED	La cuenta no existe o está deshabilitada
WRITE_CHANNEL_NOT_RESTRICTED	El canal no trae un certificado cliente válido de la misma entidad
DID_INVALID	El DID no cumple la forma <code>did:web</code>
DID_DOMAIN_MISMATCH	El DID es de otro dominio
NAMESPACE_NOT_RESERVED	Nadie ha reservado esa ruta
NAMESPACE_NOT_OWNED	La ruta tiene dueño, pero es otra entidad

12.7 La ligadura certificado↔entidad

```
$ sed -n '/fun channelRestricted/,/^    }/p' ../vdr-
service/src/main/kotlin/co/org/avance/ssi/vdr/AuthService.kt
    fun channelRestricted(client: ClientConfig, verifyHeader: String?, dnHeader: String?): Boolean {
        if (!cfg.requireMtls) return true
        val cn = mtlscn(verifyHeader, dnHeader) ?: return false
        return cn == (client.mtlscn ?: client.clientId)
    }
```

Se lee así: si se exige mTLS (en este laboratorio sí), se toma el **CN** del certificado; si no hay, el canal **no está restringido**; si hay, debe ser **igual** al nombre esperado de la entidad (`mtlscn` o, por defecto, su `clientId`). Es la base del ejercicio «usuario de uno con carné de otro» (§19.3).

El token, además, dura:

```
$ grep -rn 'tokenTtl\|TOKEN_TTL' ../vdr-service/src/main/kotlin/co/org/avance/ssi/vdr/AppConfig.kt |
cut -c1-160
35:     val tokenTtlSeconds: Long = 600,
66:         tokenTtlSeconds = env["TOKEN_TTL_SECONDS"]?.toLong() ?: 600,
```

600 segundos = 10 minutos.

13. Mapa de requisitos a implementación

Requisito de la ERSO	Implementación	Dónde
Paso 1: precondiciones	<code>preconditionFailures</code> antes de emitir el desafío	<code>RegistryService.kt</code>
Paso 2: construir el documento	<code>DidDocumentBuilder.build</code> (con <code>service</code> opcional)	<code>did-core</code>
Paso 3: canal autenticado	<code>nginx ssl_verify_client; AuthService</code> (token, ligadura)	<code>nginx.conf</code> , <code>AuthService.kt</code>
Paso 3: permiso por espacio	Scopes <code>did:write:<espacio> + namespaceOwner</code>	<code>AuthService.kt</code> , <code>RegistryStore.kt</code>
Paso 4: confirmar	<code>RegistryService.confirm</code>	<code>RegistryService.kt</code>
Paso 5: evidencia	Tabla <code>operations</code> , <code>audit_log</code> , <code>GET /operations/{id}</code>	<code>RegistryStore.kt</code> , <code>Server.kt</code>
Condición 4: canales separados	Dos puertos <code>nginx</code> y dos servidores <code>Ktor</code>	<code>nginx.conf</code> , <code>Server.kt</code>
Condición 5: sin secretos	<code>DidDocumentValidator</code> (lista blanca)	<code>did-core</code>

PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA PASO Y CADA CRITERIO

14. Cómo funcionan los laboratorios

Cada laboratorio tiene la misma estructura:

1.  **Predice**: una pregunta antes de ejecutar (con su respuesta en §24).
2. **Comandos**, cada uno con su respuesta real y su explicación: *qué hace, qué se busca y qué significa* lo obtenido.
3.  **Conclusión**: la evidencia que se lleva al criterio.

Lab	Prueba...	Sección
A	Paso 1 — condiciones (y, con ellas, parte del criterio 2)	§16
B	Paso 2 — construir el documento institucional	§17
C	Criterio 1 — se sirve por HTTPS y es resoluble	§18
D	Criterio 2 — escritura autenticada, trazada y limitada	§19
E	Criterio 3 — sin claves privadas ni datos civiles	§20
F	Criterio 4 — evidencia de publicación registrada	§21
G	Pruebas automáticas y recorrido completo	§22

⚠ Los laboratorios son **una sola sesión continua**: A y B preparan lo que usan C a F (el DID de prueba, la clave y el documento). Si se interrumpe la sesión, hay que repetir desde la preparación. El script `scripts/lab-006-completo.sh` ejecuta todo seguido.

15. Preparación común: un identificador de prueba único

Cada ejecución crea un DID nuevo bajo `entidades/inst-006-<hora>`, para poder repetir los ejercicios sin choques.

```
$ ID=inst-006-$(date +%s); DID=did:web:$D:entidades:$ID; echo "ID=$ID"; echo "DID=$DID"
ID=inst-006-1790873757
DID=did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757
```

- `date +%s` entrega los segundos transcurridos desde 1970; garantiza un nombre distinto cada vez.
- El DID resultante cumple la regla de `did:web:` el dominio, luego `:entidades:` y el nombre.

16. Laboratorio A — Paso 1: verificar las precondiciones

Qué se busca. Demostrar que **antes de hacer nada** el registro comprueba que la entidad está habilitada y que el espacio es suyo, y que cuando no lo es **rechaza el intento sin gastar un desafío**.

Predice antes de ejecutar

lab-operator (el laboratorio) intenta **empezar** a modificar el DID real de Avance (entidades/avance), con sus credenciales legítimas y su propio certificado. ¿Qué respuesta esperas? ¿Cambiará el número de desafíos guardados en la base?

16.1 Las cuentas y los espacios

```
$ echo "select client_id, display_name, enabled from entity_accounts order by 1" | $PSQL
client_id | display_name | enabled
-----+-----+-----
avance-issuer | Avance (emisor institucional) | t
lab-operator | Laboratorio de pruebas de identidad | t
vdr-admin | Administración VDR | t
wallet-backend | Wallet Backend | t
(4 rows)
$ echo "select path, owner_client_id from namespaces where path not like '%prueba-005%' order by path" | $PSQL
path | owner_client_id
-----+-----
entidades/* | avance-issuer
entidades/avance | avance-issuer
entidades/avance-ciclo | avance-issuer
entidades/inst-006-1790873626 | avance-issuer
lab/* | lab-operator
lab/laboratorio | lab-operator
titulares/* | wallet-backend
titulares/WI-RWVC8mA75544XCtyKnInxiAFrvKaR-r6WjmDHw6w | wallet-backend
(8 rows)
```

✓ avance-issuer está habilitada y es dueña de entidades/avance, entidades/avance-ciclo y entidades/*. El laboratorio solo es dueño de lab/... Esto responde al paso 1: *cuenta habilitada, con namespace propio y ruta reservada*.

16.2 Un intento ajeno contra el DID real de Avance

Primero se cuenta cuántos desafíos hay guardados, luego se intenta, y se vuelve a contar.

```
$ echo "desafíos antes: $($PSQL -tAc "select count(*) from challenges")"
desafíos antes: 26
$ ESCRIBIR lab-operator $CLIENT_SECRET_LAB lab-operator did:web:$D:entidades:avance --purpose UPDATE
--expected 1 --key lab.json --doc lab-doc.json 2>&1 | tail -n +2
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío:
precondiciones incumplidas","details":["NAMESPACE_NOT_OWNED"]}
$ echo "desafíos después: $($PSQL -tAc "select count(*) from challenges")"
desafíos después: 26
```

- **Qué hace.** ESCRIBIR lab-operator ... ejecuta el protocolo completo con las credenciales del laboratorio, apuntando a did:web:...:entidades:avance.

- **Qué se busca.** Que se detenga en el paso 2 (desafío) y que **no consuma ningún desafío**.
- **Qué significa.** Se obtuvo `412 PRECONDITION_FAILED` con `NAMESPACE_NOT_OWNED`: el token del laboratorio es válido (entró bien), pero la ruta pertenece a otro. El contador sigue igual (`26 → 26`): la petición se rechazó **antes** de emitir desafío alguno.

16.3 Quedó anotado

```
$ echo "select action, detail->'failures' as motivos from audit_log where action = 'CHALLENGE_DENIED'
order by id desc limit 3" | $PSQL
  action      |      motivos
-----+-----
CHALLENGE_DENIED | ["NAMESPACE_NOT_OWNED"]
CHALLENGE_DENIED | ["NAMESPACE_NOT_OWNED"]
CHALLENGE_DENIED | ["NAMESPACE_NOT_OWNED"]
(3 rows)
```

✓ Cada rechazo deja una fila `CHALLENGE_DENIED` con el **motivo exacto**. Las filas recientes corresponden a este intento y a ensayos previos del mismo tipo; si se consultan más filas hacia atrás aparecen otros motivos de pruebas anteriores (`DID_INVALID`, `DID_DOMAIN_MISMATCH`).

🎯 Conclusión del laboratorio A

- El registro evalúa las **precondiciones** (cuenta, canal, DID, dominio, espacio) *antes* de emitir un desafío.
- Un intento ajeno **no deja residuos** (no consume desafíos) pero **sí deja un rastro** de auditoría.

17. Laboratorio B — Paso 2: construir el documento institucional

✍️ Predice antes de ejecutar

El documento institucional se parece al de la ERSO 005, pero con una diferencia: lleva una propiedad más. ¿Cuál crees que es y por qué sería útil para una institución emisora?

17.1 Generar la clave de la institución

```
$ T00LS keygen --out c006-clave.json
Clave P-256 generada. Multikey público: zDnaeqfPj5i6kH4Hny8fntpEeb1EygHHG6uBAddsTguUXi1BK
```

- **Qué hace.** `keygen` crea un par de claves P-256: guarda la clave **privada** (en el archivo) y escribe la **pública** en Multikey.
- **Qué significa.** `zDn...` es la clave pública en formato Multikey. El archivo `c006-clave.json` contiene también la privada: **es el equivalente del sello del notario y no debe salir de la institución**. En este laboratorio vive en `evidencias/work/`, que está excluido del repositorio.

17.2 Construir el documento, con el servicio de emisión

```
$ T00LS build-doc --did $DID --key c006-clave.json --service-url "https://$D/issuer" --out c006-  
doc.json >/dev/null; jq . $W/c006-doc.json  
{  
  "@context": [  
    "https://www.w3.org/ns/did/v1",  
    "https://w3id.org/security/multikey/v1"  
  ],  
  "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757",  
  "verificationMethod": [  
    {  
      "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757#key-1",  
      "type": "Multikey",  
      "controller": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757",  
      "publicKeyMultibase": "zDnaeqfPj5i6kH4Hny8fntpEeb1EyghHG6uBAddstguUXi1BK"  
    }  
  ],  
  "authentication": [  
    "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757#key-1"  
  ],  
  "assertionMethod": [  
    "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757#key-1"  
  ],  
  "service": [  
    {  
      "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757#issuer",  
      "type": "OID4VCI",  
      "serviceEndpoint": "https://civica-desarrollo.avance.org.co/issuer"  
    }  
  ]  
}
```

- **Qué hace.** `build-doc ... --service-url https://$D/issuer` arma el documento con la clave pública y añade un *servicio* de tipo `OID4VCI` que dice «mis credenciales se emiten en esta dirección».
- **Qué se busca.** Que contenga: `id`, `verificationMethod` (la clave pública en Multikey), `authentication`, `assertionMethod` y `service`.
- **Qué significa.** El `id` coincide con el DID; la clave está en `publicKeyMultibase`; las dos relaciones apuntan a `#key-1`; y el servicio `#issuer` apunta a una URL **https** (el validador rechaza las que no lo son).

17.3 Las propiedades y la ausencia de secretos

```
$ jq -c 'keys' $W/c006-doc.json; grep -c -iE "privateKey|pkcs8" $W/c006-doc.json  
["@context","assertionMethod","authentication","id","service","verificationMethod"]  
0
```

- **Primera línea.** Las seis propiedades de nivel superior (todas permitidas por el perfil institucional).
- **Segunda línea (0).** `grep -c` cuenta cuántas líneas contienen `privateKey` o `pkcs8`: **cero**. (El comando termina con código 1 porque `grep` considera «sin coincidencias» un resultado negativo; es lo que queremos.)

Conclusión del laboratorio B

El documento institucional contiene **solo material público** y, opcionalmente, el servicio donde se emiten credenciales.

18. CRITERIO 1 — El DID Document institucional se sirve por HTTPS y es resoluble

Criterio. "El DID Document institucional se sirve por HTTPS y es resoluble; evidencia: lectura desde la URL pública."

Qué se busca. (a) Que la lectura pública por HTTPS devuelva el documento, y (b) que un cliente DID lo **resuelva** y lo valide.

Predice antes de ejecutar

Después de publicar, ¿qué Content-Type esperas ver? ¿Y qué valor tendrá el encabezado X-Content-Hash respecto del hash que devolvió la escritura?

18.1 Publicar con las credenciales de la institución

```
$ OUT=$(ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DID --purpose CREATE --expected 0
--key c006-clave.json --doc c006-doc.json 2>&1); echo "$OUT"
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=339057f0-686b-45f1-932d-c093db2f83fe audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=CREATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:inst-006-
1790873757#key-1 docHash=sha256:9456919c8adf700f9683de8207b059f3195b953897bd29fa1201fa6b3b23816f
[4/4] escritura .... 201 If-Match=0 Idempotency-Key=c5f32598-b878-48ae-ba09-e411727cf374
{
  "operationId": "73bfa733-0cdd-41dd-ba2b-8b2a83db4b1c",
  "did": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757",
  "purpose": "CREATE",
  "status": "CONFIRMED",
  "version": 1,
  "hash": "sha256:9456919c8adf700f9683de8207b059f3195b953897bd29fa1201fa6b3b23816f",
  "publicUrl": "https://civica-desarrollo.avance.org.co/entidades/inst-006-1790873757/did.json"
}
```

- **Qué hace.** Ejecuta las cuatro fases del protocolo: [1/4] token, [2/4] desafío, [3/4] prueba de posesión firmada, [4/4] escritura. `--purpose CREATE --expected 0` significa «crear; hoy no existe (versión 0)».
- **Qué se busca.** 201 y estado `CONFIRMED`.
- **Qué significa.** El servidor guardó la versión 1 y **se leyó a sí mismo por la URL pública** (`CONFIRMED`) comprobando que el hash coincide. `publicUrl` es la dirección calculada a partir del DID.

18.2 Leer por la puerta pública (encabezados)

```
$ curl -s -D - -o /dev/null $PUB https://$D:8443/entidades/$ID/did.json
HTTP/1.1 200 OK
```

```
Server: nginx
Date: Thu, 01 Oct 2026 16:56:04 GMT
Content-Type: application/did+json
Content-Length: 823
Connection: keep-alive
ETag: "1"
Cache-Control: no-cache
X-Content-Hash: sha256:9456919c8adf700f9683de8207b059f3195b953897bd29fa1201fa6b3b23816f
X-Content-Type-Options: nosniff
```

✓ Qué mirar: 200 OK; Content-Type: application/did+json (el tipo correcto para un DID Document); X-Content-Hash **idéntico** al hash de la escritura; ETag: "1" (versión 1); Cache-Control: no-cache (los clientes deben revalidar siempre, para no ver versiones viejas); X-Content-Type-Options: nosniff.

18.3 El contenido

```
$ curl -s $PUB https://$D:8443/entidades/$ID/did.json | jq .
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/multikey/v1"
  ],
  "assertionMethod": [
    "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757#key-1"
  ],
  "authentication": [
    "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757#key-1"
  ],
  "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757",
  "service": [
    {
      "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757#issuer",
      "serviceEndpoint": "https://civica-desarrollo.avance.org.co/issuer",
      "type": "0ID4VCI"
    }
  ],
  "verificationMethod": [
    {
      "controller": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757",
      "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757#key-1",
      "publicKeyMultibase": "zDnaeqfPj5i6kH4Hny8fntpEeb1EyghHG6uBAddsTguUXi1BK",
      "type": "Multikey"
    }
  ]
}
```

El documento servido es el mismo que se construyó, escrito en forma canónica (claves en orden alfabético).

18.4 Resolverlo con un cliente DID

```
$ T00LS resolve --did $DID --ca /certs/ca.crt | grep -E "^(resolutionMetadata|documentMetadata)" |
cut -c1-200
resolutionMetadata: ResolutionMetadata(error=null, message=null, contentType=application/did+json,
url=https://civica-desarrollo.avance.org.co/entidades/inst-006-1790873757/did.json, violations=[])
documentMetadata: DocumentMetadata(deactivated=false, versionId=1,
contentHash=sha256:9456919c8adf700f9683de8207b059f3195b953897bd29fa1201fa6b3b23816f)
```

- **Qué hace.** `resolve` usa el `resolver` del proyecto (`DidWebResolver`, ERSO 007): convierte el DID en URL, la lee por HTTPS (con validación de certificado), valida el documento y devuelve metadatos.
- **Qué significa.** `error=null` y `violations=[]` indican que **lo aceptó sin reparos**; `versionId=1` y `contentHash` coinciden con la evidencia. Esto cubre la parte «es resoluble» del criterio.

18.5 El DID real de Avance también se resuelve y verifica firmas

```
$ curl -s -o /dev/null -w "DID institucional real de Avance -> HTTP %{http_code}\n" $PUB
https://$D:8443/entidades/avance/did.json; T00LS resolve --did did:web:$D:entidades:avance --ca
/certs/ca.crt | grep -E "^resolutionMetadata" | cut -c1-200
DID institucional real de Avance -> HTTP 200
resolutionMetadata: ResolutionMetadata(error=null, message=null, contentType=application/did+json,
url=https://civica-desarrollo.avance.org.co/entidades/avance/did.json, violations=[])
$ JWS=$(T00LS sign --key avance.json --kid "did:web:$D:entidades:avance#key-1" --message "firma-
institucional" | tail -1); T00LS verify --jws "$JWS" --purpose assertionMethod --did
did:web:$D:entidades:avance --ca /certs/ca.crt
VALIDA did=did:web:civica-desarrollo.avance.org.co:entidades:avance kid=did:web:civica-
desarrollo.avance.org.co:entidades:avance#key-1 payload=firma-institucional
```

- La primera orden confirma que el DID institucional **real** (entidades/avance) responde `200` y se resuelve.
- La segunda hace lo que haría un verificador: Avance **firma** un mensaje con su clave privada (`sign`), y un tercero **verifica** esa firma (`verify`) usando *solo* la clave pública que leyó del documento publicado. Resultado `VALIDA`. Es la utilidad práctica del documento: **permite comprobar una firma de la institución.**

🎯 Conclusión — evidencia del criterio 1

- Lectura pública por HTTPS con tipo `application/did+json`, hash y versión.
- El DID se resuelve con un cliente independiente, sin errores.
- Una firma de la institución se verifica con la clave publicada.

19. CRITERIO 2 — La escritura está autenticada y trazada, limitada al namespace de la entidad

Criterio. "La escritura está autenticada y trazada, limitada al namespace de la entidad; evidencia: registro de la operación."

Qué se busca. Tres cosas: **autenticada** (se prueba con los intentos que *deben fallar*), **trazada** (queda en la auditoría) y **limitada** (no se puede salir del espacio propio).

🔧 Predice antes de ejecutar

Un atacante tiene (1) la contraseña de Avance pero no un certificado; (2) la contraseña de Avance y el certificado del laboratorio; (3) un certificado de una CA falsa. ¿Cuál es el código de respuesta de cada caso? ¿Cuál de los tres no dejará rastro en la auditoría de la aplicación?

19.1 Contraseña incorrecta

```
$ DIDX=did:web:$D:entidades:$ID; echo "DID objetivo de los intentos: $DIDX"
DID objetivo de los intentos: did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757
$ ESCRIBIR avance-issuer clave-incorrecta avance-issuer $DIDX --purpose UPDATE --expected 1 --key
c006-clave.json --doc c006-doc.json 2>&1
token FALLÓ: HTTP 401 {"error":"invalid_client","message":"Credenciales de cliente inválidas"}
```

- **Qué hace.** Usa el certificado correcto de Avance, pero un `client_secret` inventado.
- **Qué significa.** 401 `invalid_client`: el certificado es válido, pero **la contraseña no**; no se emite token.

19.2 Sin certificado de cliente

```
$ curl -s -o /dev/null -w "token SIN certificado cliente -> HTTP %{http_code}\n" $ADM -X POST
https://$D:9443/admin/v1/oauth/token -d grant_type=client_credentials -d client_id=avance-issuer -d
client_secret=$CLIENT_SECRET_AVANCE
token SIN certificado cliente -> HTTP 400
```

- **Qué hace.** Pide un token a la puerta de escritura (9443) con la contraseña **correcta** pero **sin certificado**.
- **Qué significa.** 400: nginx rechaza antes de llegar a la aplicación (`ssl_verify_client on`). La contraseña, por sí sola, no basta.

19.3 Contraseña de Avance con el certificado de otra entidad

```
$ ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE lab-operator $DIDX --purpose UPDATE --expected 1 --key
c006-clave.json --doc c006-doc.json 2>&1
token FALLÓ: HTTP 403 {"error":"MTLS_REQUIRED","message":"El canal de escritura exige certificado
cliente válido de la entidad"}
```

- **Qué hace.** Usuario y contraseña de `avance-issuer`, pero presenta el certificado de `lab-operator` (válido, firmado por nuestra CA).
- **Qué significa.** 403 `MTLS_REQUIRED`: el certificado es auténtico, pero su CN (`lab-operator`) **no es la entidad del token** (`avance-issuer`). Aquí actúa la **ligadura**: un token (o contraseña) robado no sirve con un certificado ajeno.

19.4 Certificado de una CA desconocida

```
$ ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE intruso $DIDX --purpose UPDATE --expected 1 --key
c006-clave.json --doc c006-doc.json 2>&1 | head -3 | cut -c1-200
token FALLÓ: HTTP 400 <html>
<head><title>400 No required SSL certificate was sent</title></head>
<body>
```

- **Qué hace.** Presenta `intruso.p12`, un certificado firmado por una CA que nginx no conoce.
- **Qué significa.** nginx corta con 400 `No required SSL certificate was sent`. El cliente ni siquiera llega a ofrecer su certificado, porque no está firmado por ninguna de las CA que nginx declara aceptar. Es un rechazo **del portero**.
- **⚠ Consecuencia:** como la petición nunca llega a la aplicación, **no aparece en la auditoría de la aplicación**; solo en los registros de nginx (§27).

19.5 La limitación al espacio: Avance no puede salir de entidades/...

```
$ ESCRIBIR lab-operator $CLIENT_SECRET_LAB lab-operator did:web:$D:entidades:avance --purpose UPDATE
--expected 1 --key lab.json --doc lab-doc.json 2>&1 | tail -n +2
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío:
precondiciones incumplidas","details":["NAMESPACE_NOT_OWNED"]}
$ ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer did:web:$D:lab:invasion --purpose CREATE
--expected 0 --key c006-clave.json --doc c006-doc.json 2>&1 | tail -n +2
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío:
precondiciones incumplidas","details":["NAMESPACE_NOT_OWNED"]}
$ ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer did:web:$D:titulares:invasion --purpose
CREATE --expected 0 --key c006-clave.json --doc c006-doc.json 2>&1 | tail -n +2
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío:
precondiciones incumplidas","details":["NAMESPACE_NOT_OWNED"]}
```

- **Primer caso:** el laboratorio contra `entidades/avance` (idéntico al de §16).
- **Segundo caso:** Avance (con todas sus credenciales correctas) intentando escribir en `lab:invasion`, que es del laboratorio.
- **Tercer caso:** Avance intentando escribir en `titulares:invasion`, que es de la cartera.
- **Qué significa.** Los tres: 412 `NAMESPACE_NOT_OWNED`. Aunque el usuario esté autenticado perfectamente, **solo puede escribir en su espacio**. Esto prueba la condición «el namespace institucional es propio y no colisiona con el del titular».

19.6 Las puertas no se cruzan

```
$ curl -s -o /dev/null -w "PUT por la puerta pública          -> HTTP %{http_code}\n" -X PUT $PUB
https://$D:8443/entidades/avance/did.json
PUT por la puerta pública          -> HTTP 403
$ curl -s -o /dev/null -w "leer did.json por la de escritura -> HTTP %{http_code}\n" $ADM --cert
certs/avance-issuer.crt --key certs/avance-issuer.key https://$D:9443/entidades/avance/did.json
leer did.json por la de escritura -> HTTP 404
```

- Escribir (PUT) por la puerta **pública** → 403 (nginx solo deja GET / HEAD).
- Leer un `did.json` por la puerta de **escritura** → 404 (esa puerta no tiene esa ruta).

Esto cumple la condición de **canales separados**.

19.7 La trazabilidad: el libro de auditoría

```
$ ADMINISTRAR vdr-admin $CLIENT_SECRET_ADMIN vdr-admin --path "/admin/v1/audit" 2>&1 | sed -n '/^\
[/,$p' | jq -r '[-12:][ ] | "\(.at[11:23]) \(.actor) \(.action) \(.detail[0:90])"'
16:56:04.346 avance-issuer CHALLENGE_ISSUED {"purpose": "CREATE", "audience": "vdr:civica-
desarrollo.avance.org.co:did-operation", "ex
16:56:04.370 avance-issuer WRITE_CREATE {"hash":
"sha256:9456919c8adf700f9683de8207b059f3195b953897bd29fa1201fa6b3b23816f", "opera
16:56:04.393 system PUBLICATION_CONFIRMED {"url": "https://civica-
desarrollo.avance.org.co/entidades/inst-006-1790873757/did.json",
16:56:13.568 avance-issuer TOKEN_DENIED {"detail": "credenciales inválidas"}
16:56:15.544 avance-issuer TOKEN_DENIED {"detail": "canal sin mTLS válido"}
16:56:19.428 lab-operator TOKEN_ISSUED {"detail": "scopes=did:write:lab/laboratorio,
did:write:lab/*"}
16:56:19.467 lab-operator CHALLENGE_DENIED {"purpose": "UPDATE", "failures":
["NAMESPACE_NOT_OWNED"]}
16:56:21.390 avance-issuer TOKEN_ISSUED {"detail": "scopes=did:write:entidades/avance,
did:write:entidades/avance-ciclo, did:write
16:56:21.429 avance-issuer CHALLENGE_DENIED {"purpose": "CREATE", "failures":
["NAMESPACE_NOT_OWNED"]}
16:56:23.355 avance-issuer TOKEN_ISSUED {"detail": "scopes=did:write:entidades/avance,
```

```
did:write:entidades/avance-ciclo, did:write
16:56:23.393 avance-issuer CHALLENGE_DENIED {"purpose": "CREATE", "failures":
["NAMESPACE_NOT_OWNED"]}
16:56:25.306 vdr-admin TOKEN_ISSUED {"detail": "scopes="}
```

✓ **Cómo leerlo.** Cada línea: *hora, actor, acción, detalle*.

- CHALLENGE_ISSUED → WRITE_CREATE → PUBLICATION_CONFIRMED: la publicación legítima, con **actor avance-issuer**.
- TOKEN_DENIED / credenciales inválidas: la contraseña incorrecta (§19.1).
- TOKEN_DENIED / canal sin mTLS válido: el certificado ajeno (§19.3).
- TOKEN_ISSUED + CHALLENGE_DENIED / NAMESPACE_NOT_OWNED: los intentos fuera del espacio (§19.5), con el motivo.
- No aparecen el caso sin certificado (§19.2) ni el de la CA falsa (§19.4): los frenó nginx.

🎯 Conclusión — evidencia del criterio 2

Pregunta	Respuesta	Evidencia
¿Autenticada?	Tres capas: certificado cliente (nginx), credenciales + token (aplicación) y firma con la clave del DID	§19.1 a §19.4
¿Limitada?	Solo al propio espacio; fuera de él, 412 NAMESPACE_NOT_OWNED	§19.5
¿Trazada?	audit_log con actor, acción y detalle	§19.7
¿Canales separados?	Escritura por 9443, lectura por 8443	§19.6

20. CRITERIO 3 — El documento no expone material de claves privadas ni datos civiles

Criterio. "El documento no expone material de claves privadas ni datos civiles; evidencia: revisión del contenido."

Qué se busca. (a) Revisar el documento publicado y (b) comprobar que el sistema **impide** publicar documentos con ese contenido.

🔧 Predice antes de ejecutar

Si intento publicar un documento con una propiedad `credentialSubject` que contiene un nombre de persona, ¿el sistema lo rechazará por una sola razón o por varias?

20.1 Revisar el documento publicado

```
$ curl -s $PUB https://$D:8443/entidades/$ID/did.json | grep -c -iE
'private|secret|credentialSubject|birthDate|"d":'
0
$ PRIV=$(jq -r .privateKeyPkcs8 $W/c006-clave.json); curl -s $PUB
https://$D:8443/entidades/$ID/did.json | grep -c -F "$PRIV"
0
```

- **Primera orden.** Cuenta cuántas líneas del documento público contienen palabras sospechosas: `private`, `secret`, `credentialSubject`, `birthDate` o `"d":` (la «d» es el componente privado de una clave en formato JWK). Resultado **0**.
- **Segunda orden.** Busca la **clave privada exacta** que se generó (léida del archivo) dentro del documento público. Resultado **0**.
- ⚠ Ambas terminan con código 1 porque `grep` interpreta «sin coincidencias» como no-éxito; aquí es el resultado deseado.

20.2 Intentar publicar documentos prohibidos

Se prepara una base para los intentos (un DID distinto, para no tocar el publicado), y se fabrican tres documentos defectuosos:

```
$ DIDN=did:web:$D:entidades:$ID-neg; TOOLS build-doc --did $DIDN --key c006-clave.json --service-url "https://$D/issuer" --out c006-neg.json >/dev/null; echo "base de los intentos: $DIDN"
base de los intentos: did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757-neg
```

(a) Con un campo de clave privada dentro de la clave de verificación:

```
$ jq '.verificationMethod[0].privateKeyMultibase="z1234"' $W/c006-neg.json > $W/c006-neg1.json; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDN --purpose CREATE --expected 0 --key c006-clave.json --doc c006-neg1.json 2>&1 | tail -n +4
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=f3c095ef-4217-4bf7-a4a0-4a798deec8c9
{
  "error": "INVALID_DOCUMENT",
  "message": "El documento no cumple el perfil de publicación",
  "details": [
    "PRIVATE_KEY_MATERIAL: Campo prohibido con posible clave privada en $.verificationMethod[0].privateKeyMultibase"
  ]
}
```

(b) Con datos civiles (`credentialSubject` con un nombre):

```
$ jq '.credentialSubject={"name":"Ana Pérez"}' $W/c006-neg.json > $W/c006-neg2.json; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDN --purpose CREATE --expected 0 --key c006-clave.json --doc c006-neg2.json 2>&1 | tail -n +4
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=d2dd42b5-222c-43fe-af4d-c0e6df7e856b
{
  "error": "INVALID_DOCUMENT",
  "message": "El documento no cumple el perfil de publicación",
  "details": [
    "CIVIL_DATA: Campo de datos civiles no permitido en $.credentialSubject",
    "CIVIL_DATA: Campo de datos civiles no permitido en $.credentialSubject.name",
    "UNKNOWN_PROPERTY: Propiedad no permitida en el perfil de publicación: 'credentialSubject'"
  ]
}
```

(c) Con un controlador ajeno:

```
$ jq '.controller="did:web:otra.entidad"' $W/c006-neg.json > $W/c006-neg3.json; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDN --purpose CREATE --expected 0 --key c006-clave.json --doc c006-neg3.json 2>&1 | tail -n +4
[4/4] escritura .... 422 If-Match=0 Idempotency-Key=c06896cb-5cb5-4abd-9a76-c13529fad746
{
  "error": "INVALID_DOCUMENT",
  "message": "El documento no cumple el perfil de publicación",
  "details": [
```



```
}
  ]
  "UNAUTHORIZED_CONTROLLER: Controlador no autorizado: did:web:otra.entidad"
}
```

✓ Todas dan 422 INVALID_DOCUMENT con el motivo preciso. En (b) el sistema da **tres** razones: dos de datos civiles (el campo y su contenido) y una de «propiedad no permitida». Esto muestra que la **lista blanca** funciona en capas.

20.3 No quedó nada

```
$ echo "DID registrados con ese identificador: $($PSQL -tAc "select count(*) from did_documents where did = '$DIDN'")"
DID registrados con ese identificador: 0
```

El número de DID registrados con ese identificador es **0**: un documento rechazado no deja ni siquiera un rastro parcial.

🎯 Conclusión — evidencia del criterio 3

- El documento publicado no contiene material prohibido (§20.1).
- Los intentos de publicar material prohibido o un controlador ajeno se rechazan con 422 y el motivo (§20.2), y no quedan registrados (§20.3).

21. CRITERIO 4 — La evidencia de publicación (versión, hash y URL pública) queda registrada

Criterio. "La evidencia de publicación (versión, hash y URL pública) queda registrada; evidencia: registro de publicación."

Qué se busca. Consultar la operación de la publicación del laboratorio C y comprobar que guarda *versión, hash y URL*, y que el hash coincide con el del documento realmente servido.

🔧 Predice antes de ejecutar

Si consulto la operación por la API y por la base de datos, ¿deben coincidir? ¿Y el hash de la operación con el hash del documento que bajo hoy de la URL pública?

21.1 Recuperar el identificador de la operación

```
$ OPID=$(echo "$OUT" | sed -n '/^{\/, $p' | jq -r .operationId); echo "operationId de la publicación: $OPID"
operationId de la publicación: 73bfa733-0cdd-41dd-ba2b-8b2a83db4b1c
```

El `operationId` se obtuvo de la respuesta de escritura del laboratorio C (§18.1).

21.2 Por la API (la forma oficial)

```
$ ADMINISTRAR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer --path "/admin/v1/operations/$OPID" 2>&1 | sed -n '/^{\/, $p'
```



```
{
  "id": "73bfa733-0cdd-41dd-ba2b-8b2a83db4b1c",
  "did": "did:web:civica-desarrollo.avance.org.co:entidades:inst-006-1790873757",
  "purpose": "CREATE",
  "clientId": "avance-issuer",
  "status": "CONFIRMED",
  "version": 1,
  "hash": "sha256:9456919c8adf700f9683de8207b059f3195b953897bd29fa1201fa6b3b23816f",
  "publicUrl": "https://civica-desarrollo.avance.org.co/entidades/inst-006-1790873757/did.json",
  "idempotencyKey": "c5f32598-b878-48ae-ba09-e411727cf374",
  "requestHash": "sha256:71755201ea166ec0201ab27351dace6eb989328f025716894bfdc2faa7dd2f53",
  "createdAt": "2026-10-01T16:56:04.370537Z",
  "confirmedAt": "2026-10-01T16:56:04.393458Z"
}
```

- **Qué hace.** Consulta `GET /admin/v1/operations/{id}` con las credenciales y certificado de Avance (solo se puede consultar lo propio).
- **Qué significa.** Aparecen los tres datos que pide el criterio: `version: 1`, `hash` y `publicUrl`; además: quién (`clientId`), cuándo se creó y cuándo se **confirmó** (`confirmedAt`, unos 23 milisegundos después: el registro leyó su propia URL), la clave de idempotencia y el hash de la petición.

21.3 Por la base de datos

```
$ echo "select purpose, status, version, left(hash,26) as hash, public_url, (confirmed_at is not null) as confirmada from operations where id = '$0PID' " | $PSQL
 purpose | status | version | hash | public_url | confirmada
-----+-----+-----+-----+-----+-----
 CREATE | CONFIRMED | 1 | sha256:9456919c8adf700f968 | https://civica-desarrollo.avance.org.co/entidades/inst-006-1790873757/did.json | t
(1 row)
```

Los mismos datos, directamente en la tabla `operations`: estado `CONFIRMED`, versión 1, URL pública y `confirmada = t` (verdadero).

21.4 El hash registrado = el hash del documento servido

```
$ curl -s $PUB https://$D:8443/entidades/$ID/did.json | sha256sum | awk '{print "hash del documento público: sha256:"$1}'; echo "select 'hash de la evidencia' : '||hash from operations where id = '$0PID' " | $PSQL -tA
hash del documento público: sha256:9456919c8adf700f9683de8207b059f3195b953897bd29fa1201fa6b3b23816f
hash de la evidencia : sha256:9456919c8adf700f9683de8207b059f3195b953897bd29fa1201fa6b3b23816f
```

- **Qué hace.** Calcula el hash del documento **tal como lo entrega ahora** la URL pública, y muestra el hash guardado en la evidencia.
- **Qué significa.** Son **idénticos**: la evidencia describe exactamente lo que el mundo ve. Si alguien cambiara el documento por debajo, no coincidirían.

21.5 El historial de esa publicación

```
$ ADMINISTRAR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer --path "/admin/v1/audit?did=$DID" 2>&1 | sed -n '/^[/,,$p' | jq -r '[] | "\(.at[11:23]) \(.actor) \(.action) v\(.version // "-")"'
16:56:04.346 avance-issuer CHALLENGE_ISSUED v-
16:56:04.370 avance-issuer WRITE_CREATE v1
16:56:04.393 system PUBLICATION_CONFIRMED v1
```

Tres eventos: se emitió el desafío, se escribió la versión 1 y el sistema confirmó la publicación.

⚠ Las tablas `audit_log` y `did_document_versions` están protegidas con **disparadores** que impiden modificarlas o borrarlas (ERSo 004), por lo que esta evidencia no se puede reescribir a posteriori.

🎯 Conclusión — evidencia del criterio 4

La publicación quedó registrada con **versión 1**, hash `sha256:...`, **URL pública** y **marca de confirmación**; el hash coincide con el del documento servido.

22. Pruebas automáticas y recorrido de extremo a extremo

22.1 Ejecutar las pruebas de la ERSo 006

La ERSo tiene su propia clase de pruebas, que ejecuta cada criterio dentro de una base de datos desechable (`vdr-test-pg`, puerto 55432):

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect
$ TEST_DB_URL=jdbc:postgresql://127.0.0.1:55432/wallet_test ./gradlew :vdr-service:test --tests '*Erso006*' --rerun-tasks -q 2>&1 | tail -3; echo "(sin salida = todas pasaron)"
(sin salida = todas pasaron)
$ python3 - <<'EOF'
> import re, glob, html
> f = glob.glob('vdr-service/build/test-results/test/*Erso006*.xml')[0]
> s = open(f, encoding='utf-8').read()
> tot = re.search(r'tests="(\d+)" skipped="(\d+)" failures="(\d+)"', s).groups()
> print(f"resultado: {tot[0]} pruebas, {tot[1]} omitidas, {tot[2]} fallos\n")
> for m in re.finditer(r'<testcase name="([\^"]+)"[\^>]*?(</>|>.(.*?)</testcase>)', s, re.S):
>     name = html.unescape(m.group(1)).removesuffix("(")
>     estado = "FALLA" if "<failure" in (m.group(3) or "") else "PASA "
>     print(f" [{estado}] {name}")
> EOF
resultado: 6 pruebas, 0 omitidas, 0 fallos

[PASA ] criterio 3 - el documento publicado no expone claves privadas ni datos civiles
[PASA ] criterio 2 - la escritura esta autenticada, trazada y limitada al namespace de la entidad
[PASA ] canales separados - el canal publico no escribe y el de escritura no sirve documentos
[PASA ] una entidad no puede colisionar con el namespace de otra
[PASA ] criterio 4 - la evidencia de publicacion version hash y URL queda registrada
[PASA ] criterio 1 - el documento institucional se sirve por el canal publico y es resoluble
```

- **Qué hace.** `./gradlew :vdr-service:test --tests '*Erso006*' --rerun-tasks` ejecuta solo las pruebas de esta ERSo (`--rerun-tasks` fuerza a no reutilizar resultados anteriores); el script en Python lista los resultados.
- **Qué significa.** Seis pruebas, ninguna falla. Cada nombre indica el criterio que cubre.

🔗 Preguntas (con respuesta)

1. *¿Por qué la prueba de «canales separados» no se ejecuta contra la base real sino contra la desechable?* → Porque las pruebas crean y borran datos; la base real contiene la evidencia de los laboratorios.
2. *¿Qué prueba cubre que dos entidades no colisionen?* → «una entidad no puede colisionar con el namespace de otra».

22.2 El recorrido de extremo a extremo (extracto del registro)

El script `scripts/e2e.sh` ejecuta, en una sola corrida, **todo el proyecto** (121 comprobaciones). Este es el bloque correspondiente a la ERSO 006:

```
ERSo 2026-006 · Publicación del DID Document institucional (entidad Avance)

Clave P-256 generada. Multikey público: zDnaegWdP7UT2CWTPx8dk5S8EtggrMUuN6dKmPhXdmHGZgnsE
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=77e705b7-d422-46bb-9de5-213e76c7a101 audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=CREATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance#key-1
docHash=sha256:2ce8444ae71f931f273979e362cdf505baca26b96ce88a4f0b5a3879e6...
[4/4] escritura .... 201 If-Match=0 Idempotency-Key=3efal602-14f8-439d-8720-1176f6937c82
{
  "operationId": "18335e26-7ef2-4b51-b76d-c30166db6f7d",
  "did": "did:web:civica-desarrollo.avance.org.co:entidades:avance",
  "purpose": "CREATE",
  "status": "CONFIRMED",
  "version": 1,
  "hash": "sha256:2ce8444ae71f931f273979e362cdf505baca26b96ce88a4f0b5a3879e6cb8f3c",
  "publicUrl": "https://civica-desarrollo.avance.org.co/entidades/avance/did.json"
}
✓ PASS C1: el documento institucional se sirve por HTTPS y es resoluble
✓ PASS C1: resoluble por el cliente consumidor (ERSo 007)
✓ PASS C3: sin claves privadas ni datos civiles
{
  "id": "18335e26-7ef2-4b51-b76d-c30166db6f7d",
  "did": "did:web:civica-desarrollo.avance.org.co:entidades:avance",
  "purpose": "CREATE",
  "clientId": "avance-issuer",
  "status": "CONFIRMED",
  "version": 1,
  "hash": "sha256:2ce8444ae71f931f273979e362cdf505baca26b96ce88a4f0b5a3879e6cb8f3c",
  "publicUrl": "https://civica-desarrollo.avance.org.co/entidades/avance/did.json",
  "idempotencyKey": "3efal602-14f8-439d-8720-1176f6937c82",
  "requestHash": "sha256:8038ec03dd2ffa13511c64c382a90183503b2cd72cf233edde96890f95871ed4",
  "createdAt": "2026-09-30T14:21:02.547095Z",
  "confirmedAt": "2026-09-30T14:21:02.570404Z"
}
✓ PASS C4: evidencia registrada (versión, hash y URL pública)
- escritura autenticada y limitada al namespace -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 409 {"error":"ALREADY_EXISTS","message":"El DID ya existe (versión 1); use
UPDATE"}
token FALLÓ: HTTP 401 {"error":"invalid_client","message":"Credenciales de cliente inválidas"}
✓ PASS C2: credenciales incorrectas -> 401 invalid_client
token FALLÓ: HTTP 403 {"error":"MTLS_REQUIRED","message":"El canal de escritura exige certificado
cliente válido de la entidad"}
✓ PASS C2: certificado de OTRA entidad con credenciales de Avance -> 403 MTLS_REQUIRED
</body>
</html>
✓ PASS C2: certificado de una CA desconocida -> handshake TLS rechazado por nginx (no llega a la
aplicación)
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 412 {"error":"PRECONDITION_FAILED","message":"No se emite desafío:
precondiciones incumplidas","details":["NAMESPACE_NOT_OWNED"]}
✓ PASS C2: laboratorio no puede escribir en el namespace de Avance (412 NAMESPACE_NOT_OWNED)
✓ PASS canal público NO acepta escrituras (PUT/POST/DELETE)
✓ PASS el canal de escritura exige certificado cliente (nginx responde 400 antes de llegar a la
aplicación)
Traza de auditoría (acciones):
2026-09-30T14:20:47.766291Z system NAMESPACE_RESERVED v-
2026-09-30T14:21:02.521336Z avance-issuer CHALLENGE_ISSUED v-
```

```
2026-09-30T14:21:02.547095Z  avance-issuer WRITE_CREATE v1
2026-09-30T14:21:02.570404Z  system PUBLICATION_CONFIRMED v1
2026-09-30T14:21:16.407928Z  lab-operator CHALLENGE_DENIED v-
✓ PASS C2: la operación quedó trazada (WRITE_CREATE con actor avance-issuer)
```

ERSo 2026-008 · Ciclo de vida y trazabilidad (namespace entidades/avance-ciclo)

```
Clave P-256 generada. Multikey público: zDnaecetVC6EQRZz6jwBpD2yRezRjoauvSHWsguvSpNGXEKK7
Clave P-256 generada. Multikey público: zDnaeU8qaBcG4Hk4o9PNDgXfSJzVwZrauzXCTFiYk3X3NsWpV
- versión 1: CREATE -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=958618fe-la9d-462a-9449-7ae815c98338 audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=CREATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance-ciclo#key-1
docHash=sha256:7909c893d0d82e5b7cb376d173d19c9e6620e0f9449b3206747f...
[4/4] escritura .... 201 If-Match=0 Idempotency-Key=68b91e32-62e5-443e-ac6c-4e4c7b4793e1
{
  ✓ PASS C4: la escritura devolvió versión, hash y URL pública
  ✓ PASS C5: confirmada por lectura de la URL y comparación de hash
  - versión 2: UPDATE con rotación de clave (firma la clave VIGENTE c1) -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=470c36a2-6727-4d38-8239-e83853b026ed audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=UPDATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance-ciclo#key-1
docHash=sha256:83d7bfea0ecebdbc2038381891c04e27c09e7c49a884afb77a...
[4/4] escritura .... 200 If-Match=1 Idempotency-Key=a0e14790-7a0a-4807-a95d-5d8f101d9d03
{
  ✓ PASS rotación aceptada: la prueba la firmó la clave vigente
  "error": "INVALID_PROOF",
  "message": "Prueba de posesión inválida: firma no válida"
}
  ✓ PASS la clave anterior ya no puede escribir (403 INVALID_PROOF)
  "currentVersion=2"
}
}
  ✓ PASS C4: versión esperada equivocada se rechaza (412 VERSION_CONFLICT)
  - versión 3: UPDATE (firma c2) -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=f1a7b890-7344-4c84-9b69-502ec3c013ab audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=UPDATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance-ciclo#key-1
docHash=sha256:eb337892c97281c6a189a5231eda641efe3883c431cb7ealed02...
[4/4] escritura .... 200 If-Match=2 Idempotency-Key=9d04e3bf-4ca0-4c4f-9181-c92e04c189e6
{
  ✓ PASS versión 3 publicada
  - versión 4: DEACTIVATE -
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío ..... 201 id=5a2d3beb-b2f1-4d73-911c-955565f96dc3 audience=vdr:civica-
desarrollo.avance.org.co:did-operation purpose=DEACTIVATE
[3/4] prueba ..... ES256 kid=did:web:civica-desarrollo.avance.org.co:entidades:avance-ciclo#key-1
docHash=sha256:eb337892c97281c6a189a5231eda641efe3883c431cb7ealed02...
[4/4] escritura .... 200 If-Match=3 Idempotency-Key=e0924ccf-c437-4044-8e13-9d2336f54f5e
{
  ✓ PASS desactivación confirmada (la URL pública responde 410)
  ✓ PASS lectura pública del DID desactivado -> 410 Gone
[1/4] token ..... OK (client_credentials + mTLS)
[2/4] desafío FALLÓ: HTTP 409 {"error":"TERMINAL_STATE","message":"El historial está desactivado; se
requiere una nueva instancia (otro namespace)"}
  ✓ PASS regla transversal: un historial terminal no vuelve a activarse (409 TERMINAL_STATE)
  Historial de versiones (C7):
    v1 CREATE 2026-09-30T14:21:26.800033Z
sha256:7909c893d0d82e5b7cb376d173d19c9e6620e0f9449b3206747f7ea927af0d77
    v2 UPDATE 2026-09-30T14:21:30.817406Z
sha256:83d7bfea0ecebdbc2038381891c04e27c09e7c49a884afb77aaef417d1d6f0
    v3 UPDATE 2026-09-30T14:21:36.790936Z
sha256:eb337892c97281c6a189a5231eda641efe3883c431cb7ealed0298cfdadbe0f4
```

```
v4 DEACTIVATE 2026-09-30T14:21:38.868470Z
sha256:eb337892c97281c6a189a5231eda641efe3883c431cb7ea1ed0298cfdadbe0f4
✓ PASS C7: 4 versiones en orden CREATE, UPDATE, UPDATE, DEACTIVATE
✓ PASS C7: estado reconstruido en el instante de la v2 = ACTIVE, versión 2
✓ PASS C7: estado reconstruido en el instante de la v4 = DEACTIVATED
✓ PASS C7: el historial no se puede reescribir (trigger append-only en PostgreSQL)
✓ PASS C2: desafíos emitidos quedaron registrados (tipo y audiencia)
✓ PASS C1: sin certificado cliente no se emite desafío (nginx 400; la aplicación ni se entera)
✓ PASS C1: la denegación por precondiciones (namespace ajeno) dejó rastro y no creó desafío
```

PARTE V — PREGUNTAS Y RESPUESTAS

23. Preguntas sobre los conceptos

- 1. ¿Qué diferencia hay entre esta ERSO y la 005?** La 005 publica documentos de *laboratorio*; la 006, el de la **entidad institucional**, con espacio exclusivo y con servicio de emisión. La técnica es la misma; cambia el sujeto y el énfasis: autenticación, límites y evidencia.
- 2. ¿Por qué se necesitan tres pruebas para escribir (certificado, token, firma)?** Cada una responde a una pregunta distinta: el certificado, *¿quién llega por la puerta?*; el token, *¿qué puede hacer?*; la firma con la clave del DID, *¿posee realmente la clave del documento que va a publicar?* Si faltara una, un atacante con solo las otras dos podría publicar.
- 3. ¿Qué es la ligadura certificado↔token y qué ataque evita?** El CN del certificado debe ser la misma entidad que la del token. Evita que alguien con credenciales robadas de Avance las use con **su propio** certificado válido (por ejemplo, el del laboratorio).
- 4. ¿Por qué nginx sobrescribe las cabeceras X-SSL-Client-*?** La aplicación se fía de ellas para saber quién es el cliente. Si nginx las dejara pasar, un cliente podría enviar una cabecera falsa que dijera «soy avance-issuer». Como nginx las fija siempre con lo que él verificó, no se pueden falsificar; además la aplicación no es accesible directamente.
- 5. ¿Qué es un namespace y por qué es «propio»?** Una ruta reservada a una sola entidad. Es propio porque en la tabla `namespaces` cada ruta tiene un único dueño.
- 6. ¿Qué significa el comodín `entidades/*`?** Reserva un nivel: `entidades/<cualquier-nombre>`. No cubre `entidades/a/b`. Se añadió para que los laboratorios puedan repetirse sin tocar el DID real de la institución.
- 7. ¿Por qué se evalúan las precondiciones antes del desafío?** Para no gastar recursos (ni ofrecer información o superficie de ataque) con quien no tiene derecho; y para dejar claro, en la auditoría, **por qué** se rechazó.
- 8. ¿Qué diferencia hay entre «se sirve» y «es resoluble»?** «Se sirve»: el servidor entrega el JSON. «Es resoluble»: un cliente DID puede convertir el DID en ese documento y **validarlo**. Un servidor puede servir algo que el cliente no acepte (por ejemplo, un `id` que no coincide).
- 9. ¿Por qué el servicio `OID4VCI` debe usar `https`?** Porque indica a otros dónde obtener credenciales; si fuera `http`, un intermediario podría alterar la respuesta. El validador rechaza otros esquemas.
- 10. ¿Qué es la evidencia de publicación y por qué incluye el hash?** Versión + hash + URL. El hash es la huella del contenido; permite comprobar después que lo que hay publicado es exactamente lo registrado.
- 11. ¿Se puede borrar o alterar la auditoría?** No por el camino normal: hay disparadores en la base de datos que lo impiden (ERSO 004). Un administrador de la base de datos con acceso

total podría quitarlos; esa es una limitación honesta (§27).

12. ¿Qué pasa si un token se roba? Dura 10 minutos y está ligado al certificado de la entidad: solo sirve junto a ese certificado. No hay revocación anticipada (§27).

24. Preguntas por laboratorio y por criterio

Paso 1 — Precondiciones (laboratorio A)

P1. El laboratorio intenta empezar a escribir en `entidades/avance`. ¿Qué respuesta y cambia el número de desafíos? → `412 PRECONDITION_FAILED` con `NAMESPACE_NOT_OWNED`. **No** cambia el contador (26→26): se rechaza antes de emitir el desafío. Queda una fila `CHALLENGE_DENIED` en la auditoría.

P2. ¿Por qué la respuesta es 412 y no 401 o 403? → 401/403 hablan de «quién eres» o «no se te permite»; aquí la identidad es válida y lo que falla es una **precondición** del recurso (que el espacio sea tuyo). 412 = *Precondition Failed*.

Paso 2 — Construir el documento (laboratorio B)

P3. ¿Qué propiedad extra lleva el documento institucional? → `service` (tipo `OID4VCI`), que dice dónde se emiten credenciales. Útil porque quien quiera pedir credenciales a la institución puede **descubrir** su dirección a partir del DID.

P4. ¿Por qué el `grep -c` termina con código 1 aunque «salga bien»? → `grep` devuelve 1 cuando no encuentra coincidencias. Que no haya claves privadas es el resultado deseado, pero para el intérprete de comandos es «no encontrado».

Criterio 1 (laboratorio C)

P5. ¿Qué `Content-Type` y qué `X-Content-Hash`? → `application/did+json` y el **mismo** hash que devolvió la escritura.

P6. ¿Qué prueba la firma `VALIDA` de §18.5? → Que la clave pública del documento publicado verifica una firma hecha con la clave privada de Avance: el documento sirve para lo que existe.

P7. ¿Por qué `Cache-Control: no-cache`? → Para que los clientes revaliden siempre y no usen una versión caducada (por ejemplo, tras una rotación de claves).

Criterio 2 (laboratorio D)

P8. ¿Qué código da cada caso?

Caso	Código	Quién lo da
Contraseña incorrecta, certificado correcto	401 <code>invalid_client</code>	Aplicación
Contraseña correcta, sin certificado	400	nginx
Contraseña de Avance + certificado de otra entidad	403 <code>MTLS_REQUIRED</code>	Aplicación (ligadura)

Caso	Código	Quién lo da
Certificado de CA desconocida	400 (sin certificado válido)	nginx
Avance escribiendo fuera de su espacio	412 <code>NAMESPACE_NOT_OWNED</code>	Aplicación (precondición)
PUT por la puerta pública	403	nginx
Leer <code>did.json</code> por la puerta de escritura	404	nginx

P9. ¿Cuál de los intentos no aparece en la auditoría de la aplicación? → Los de nginx: **sin certificado y CA desconocida**, porque la aplicación nunca los ve.

P10. ¿Por qué `lab:invasion` y `titulares:invasion` se rechazan igual que `entidades:avance` para el laboratorio? → Porque la regla es la misma: el espacio tiene dueño y no eres tú. La dirección del intruso (Avance hacia el laboratorio, o el laboratorio hacia Avance) no importa.

Criterio 3 (laboratorio E)

P11. El documento con `credentialSubject` dio tres razones. ¿Por qué no una? → Porque la validación mira en capas: el **campo** `credentialSubject` es civil (1), su **contenido** `name` también (2) y además la propiedad **no figura** en la lista blanca (3).

P12. ¿Por qué `UNAUTHORIZED_CONTROLLER` para `did:web:otra.entidad`? → El validador solo admite como controlador al propio DID del documento. Un controlador ajeno permitiría que otro DID gobernara el documento.

P13. ¿Qué significa que `did_documents` tenga 0 filas con ese identificador? → Que un rechazo es total: no queda ni un documento parcial.

Criterio 4 (laboratorio F)

P14. ¿Qué tres datos pide el criterio y dónde están? → Versión, hash y URL pública, en `GET /operations/{id}` y en la tabla `operations`.

P15. ¿Qué demuestra que los hashes sean idénticos (§21.4)? → Que la evidencia describe exactamente lo que se sirve hoy. Si se alterara el documento, los hashes diferirían.

P16. ¿Quién puede consultar una operación? → Solo la entidad que la hizo, o la cuenta administradora; las demás reciben `403 FORBIDDEN`.

Pruebas automáticas

P17. ¿Cuántas pruebas hay para esta ERSO y qué cubren? → Seis: criterio 1, criterio 2, criterio 3, criterio 4, canales separados y no colisión de espacios.

PARTE VI — CÓMO RESPONDER A CADA CRITERIO

25. Redacción modelo de las cuatro respuestas

Criterio 1 — El DID Document institucional se sirve por HTTPS y es resoluble

El documento de la entidad `did:web:civica-desarrollo.avance.org.co:entidades:avance` se entrega por HTTPS en `https://civica-desarrollo.avance.org.co/entidades/avance/did.json` con `Content-Type: application/did+json`, `ETag` de versión y `X-Content-Hash` igual al hash registrado. El cliente DID del proyecto (`tools resolve`) lo resuelve sin violaciones, con `versionId` y `contentHash` coherentes. Además, una firma hecha con la clave privada de Avance se verifica (`VALIDA`) usando solo la clave publicada. **Evidencia:** §18.1 a §18.5.

Criterio 2 — La escritura está autenticada y trazada, limitada al namespace

La escritura solo es posible con tres pruebas simultáneas: certificado de cliente firmado por nuestra CA (nginx), credenciales de servicio que producen un token con permiso `did:write: <espacio>` y firma ES256 con la clave del propio DID sobre un desafío de un solo uso. Se comprobó que la contraseña incorrecta da 401, que sin certificado da 400, que una contraseña de Avance con el certificado de otra entidad da 403 `MTLS_REQUIRED` y que un certificado de una CA desconocida es rechazado por nginx. Avance no puede escribir fuera de su espacio (`412 NAMESPACE_NOT_OWNED` sobre `lab: ni titulares:`), y el laboratorio no puede escribir en `entidades:avance`. Todo queda en `audit_log` con actor, acción y motivo. La lectura (8443) y la escritura (9443) son canales separados. **Evidencia:** §16 y §19.

Criterio 3 — No expone claves privadas ni datos civiles

El documento publicado contiene solo propiedades de la lista blanca (`@context`, `id`, `verificationMethod`, `authentication`, `assertionMethod`, `service`). La revisión del contenido no halla palabras sospechosas ni la clave privada generada. Los intentos de publicar `privateKeyMultibase`, `credentialSubject` (con nombre) o un controlador ajeno se rechazan con 422 y su motivo, sin dejar rastro en el registro. **Evidencia:** §20.

Criterio 4 — La evidencia de publicación queda registrada

Cada publicación genera una operación con versión, hash, URL pública y marca de confirmación, consultable por `GET /admin/v1/operations/{id}` y en la tabla `operations`; la auditoría registra `CHALLENGE_ISSUED` → `WRITE_CREATE` → `PUBLICATION_CONFIRMED`. El hash de la

operación coincide con el del documento servido. Las tablas de versiones y auditoría están protegidas contra modificación. **Evidencia:** §21.

26. Lista de comprobación para quien acepta

☐ Qué comprobar	Cómo	Resultado esperado
☐ La extensión está encendida	<code>curl .../health</code>	<code>{"status":"UP","vdr":"enabled"}</code>
☐ Avance tiene cuenta habilitada y espacio	<code>select ... from entity_accounts / namespaces</code>	<code>avance-issuer</code> , <code>entidades/avance</code>
☐ Publicación legítima	ESCRIBIR <code>avance-issuer ...</code>	201 y CONFIRMED
☐ Lectura pública	<code>curl -D - .../did.json</code>	200, <code>application/did+json</code> , hash igual
☐ Resolución	<code>tools resolve</code>	<code>error=null</code> , <code>violations=[]</code>
☐ Firma verificable	<code>sign + verify</code>	VALIDA
☐ Contraseña mala	ESCRIBIR <code>... clave-incorrecta</code>	401 <code>invalid_client</code>
☐ Sin certificado	<code>curl ... :9443/oauth/token</code>	400
☐ Certificado de otra entidad	ESCRIBIR <code>... lab-operator</code>	403 <code>MTLS_REQUIRED</code>
☐ CA desconocida	ESCRIBIR <code>... intruso</code>	Rechazo de nginx
☐ Fuera del espacio	Escribir en <code>lab: / titulares: / entidades:avance</code> desde otra	412 <code>NAMESPACE_NOT_OWNED</code>
☐ Canales separados	PUT por 8443 / GET por 9443	403 / 404
☐ Sin material privado	<code>grep -c ...</code>	0
☐ Documentos prohibidos	tres variantes	422 <code>INVALID_DOCUMENT</code>
☐ Evidencia de la operación	<code>GET /operations/{id}</code>	versión, hash, URL, <code>confirmedAt</code>
☐ Hash = documento servido	<code>sha256sum</code> vs evidencia	Idénticos
☐ Auditoría	<code>GET /admin/v1/audit</code>	Eventos con actor
☐ Pruebas automáticas	<code>./gradlew :vdr-service:test --tests '*Erso006*'</code>	6/6

PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN

27. Lo que este informe NO demuestra, y lo que se encontró

27.1 Límites

Límite	Detalle
Secretos de laboratorio	Los secretos de cliente y los certificados están en <code>deploy/.env</code> y <code>deploy/certs/</code> (excluidos de git). En producción: gestor de secretos y rotación.
CA propia	La CA de laboratorio firma los certificados cliente. En producción debe ser una CA privada distinta de la del certificado público del canal de lectura.
Intentos frenados por nginx	Sin certificado o con CA desconocida: no llegan a la aplicación; quedan solo en los logs de nginx.
Sin limitación de tasa	No hay bloqueo por intentos fallidos repetidos.
Tokens sin revocación anticipada	Duran 10 minutos; no se pueden revocar antes.
did:web depende del dominio	Quien controle el DNS/dominio controla lo que se sirve.
Modo de clave	La clave de Avance es un archivo (<code>avance.json</code>). En producción debería estar en un módulo de seguridad (HSM).
Espacio entidades/*	Se reservó para repetir laboratorios; en producción conviene reservar solo rutas concretas.

27.2 Hallazgo heredado: HEAD responde 404

Descubierto en la ERSo 005 (§27.1 de su informe): nginx permite `HEAD` , pero la aplicación no tiene ruta `HEAD` , por lo que responde 404 aunque `GET` funcione. Afecta también a los documentos institucionales. No altera los criterios (que piden lectura con `GET`), pero conviene corregirlo.

28. Si algo no sale como en el informe

Síntoma	Causa probable	Qué hacer
412 <code>NAMESPACE_NOT_OWNED</code> al publicar en <code>entidades/inst-006-...</code>	<code>entidades/*</code> no está reservado (el <code>.env</code> es anterior al cambio)	Añadir <code>"entidades/*"</code> a namespaces de <code>avance-issuer</code> en <code>VDR_CLIENTS</code> y reiniciar <code>vdr</code>
401 <code>invalid_client</code> con la contraseña correcta	Variable <code>CLIENT_SECRET_AVANCE</code> sin cargar	<code>set -a; . ./env; set +a</code>

Síntoma	Causa probable	Qué hacer
403 MTLS_REQUIRED en una escritura legítima	Certificado y cuenta no coinciden (-p12 equivocado)	Usar <code>avance-issuer.p12</code> con <code>avance-issuer</code>
400 de nginx en la escritura	Falta certificado o expiró	Regenerar con <code>scripts/gen-dev-certs.sh</code>
428/412 al escribir	<code>--expected</code> no coincide con la versión actual	Consultar la versión y repetir
PENDING en vez de CONFIRMED	El registro no pudo leer su URL pública	<code>POST /admin/v1/operations/{id}/reconcile</code>
<code>jq: command not found</code> / <code>docker compose</code> falla	Falta la herramienta o los servicios no están arriba	Instalar o <code>docker compose up -d</code>
La prueba automática no conecta	<code>vdr-test-pg</code> no está levantada	Levantar la base de pruebas (puerto 55432)

29. Operación diaria

```

cd deploy
docker compose up -d          # levantar
docker compose ps             # estado
docker compose --profile tools down  # bajar (NUNCA con -v: borra la base)

```

- **Rotar la clave institucional:** generar una clave nueva, construir el documento con ella y publicar `UPDATE` con `--expected <versión actual>`. La versión anterior queda en el historial (ERSo 008 trata el ciclo de vida).
- **Dar de alta una entidad nueva:** añadir su cuenta, secreto y espacios en `VDR_CLIENTS`, generar su certificado, reiniciar `vdr`.

ANEXOS

Anexo A — Los scripts

Script	Para qué sirve
<code>scripts/lab-006-completo.sh</code>	Ejecuta los laboratorios A a G en una sola sesión y produce la salida de este informe
<code>scripts/e2e.sh</code>	Recorrido de extremo a extremo de todo el proyecto (121 comprobaciones)
<code>scripts/gen-env.sh</code>	Genera <code>deploy/.env</code> (cuentas, secretos y espacios)
<code>scripts/gen-dev-certs.sh</code>	Genera la CA y los certificados de laboratorio
<code>scripts/generar-pdf.sh</code>	Convierte un informe a PDF

Uso: `bash scripts/lab-006-completo.sh`.

Anexo B — Autocomprobación (con respuestas)

1. *¿Qué tres pruebas se necesitan para escribir?* → Certificado de cliente, token con permiso sobre el espacio, firma con la clave del DID.
2. *¿Cuál es el código si se usa la contraseña de Avance con el certificado del laboratorio?* → `403 MTLS_REQUIRED`.
3. *¿Qué pasa si Avance intenta escribir en `lab:x`?* → `412 NAMESPACE_NOT_OWNED`, sin desafío.
4. *¿Qué puerto es de lectura y cuál de escritura (en este equipo)?* → 8443 lectura; 9443 escritura.
5. *¿Por qué nginx sobrescribe `X-SSL-Client-*`?* → Para que no puedan falsificarse.
6. *¿Qué contiene la evidencia de publicación?* → Versión, hash, URL pública (y confirmación).
7. *¿Qué pasa con un documento con `credentialSubject`?* → `422 INVALID_DOCUMENT` y no se guarda nada.
8. *¿Qué diferencia hay entre `CONFIRMED` y `PENDING`?* → Si el registro pudo leer su URL pública y el hash coincidió.
9. *¿Por qué `entidades/*`?* → Para repetir laboratorios sin tocar el DID real.
10. *¿Qué intentos no deja rastro en la auditoría de la aplicación?* → Sin certificado y CA desconocida.

Anexo C — Ejercicios del método (con respuesta modelo)

Ejercicio 1. *Descompón el criterio 2.* → Verbos: *autenticada, trazada, limitada*. Evidencia: registro de la operación. Casos negativos: contraseña, certificado, espacio. Mecanismos: mTLS, OAuth2, ligadura, namespaces, auditoría.

Ejercicio 2. *Diseña una prueba para «un token robado no sirve».* → Obtener un token de Avance y presentarlo con el certificado del laboratorio: debe dar `403 MTLS_REQUIRED` (el laboratorio de §19.3 lo hace con la contraseña; la ligadura también se aplica a las llamadas con token).

Ejercicio 3. *¿Qué evidencia mostrarías si te piden demostrar «quién publicó la versión 1»? → La operación (`clientId`) y la fila `WRITE_CREATE` de auditoría (actor).*

Anexo D — Referencias

- W3C — Decentralized Identifiers (DIDs) v1.1.
- Especificación del método `did:web`.
- Multikey y multibase.
- RFC 6749 §4.4 (OAuth2 client credentials); RFC 8705 (OAuth2 mutual-TLS).
- `docs/VERSIONES-NORMATIVAS.md`, `docs/MARCO-CONCEPTUAL.md`, `docs/COMO-SE-RESOLVIO-CADA-CRITERIO.md`.